

Alternating Sponge: A Low-Memory Hash Function with Beyond-Birthday-Bound Security

Ziyang Luo¹, Yaobin Shen¹, Hailun Yan², Lei Wang³ and Dawu Gu³

¹ School of Informatics, Xiamen University, Xiamen, China
ziyangluo@stu.xmu.edu.cn, yaobin.shen@xmu.edu.cn

² School of Cryptology, University of Chinese Academy of Sciences, Beijing 100049, China

³ Shanghai Jiao Tong University, Shanghai, China

Abstract. An improved low-memory hash function scheme called Alternating Sponge (ASP) is proposed to achieve the beyond-birthday-bound security with respect to the capacity c . The scheme redesigns the state layout and permutation mechanism of double sponge construction, effectively reducing the total state space from $2r + 2c$ bits to $2r + c$ bits and increasing the number of bits squeezed per round from r to $2r$. The scheme introduces a two-phase serial permutation sequence to update the shared state, achieving a complete state transition without increasing the size of the permutation function. A security proof is provided within the indistinguishability framework, demonstrating that the scheme can achieve security comparable to the double sponge construction, while operating with a reduced state size and an increased number of squeezed bits. Overall, the design ensures the compactness and diffusion of each round of processing through layered absorption and alternating permutations, providing a solution for deploying high-security hash functions in memory-constrained environments.

Keywords: permutation-based hashing · double sponge · low memory · indistinguishability · lightweight cryptography

1 Introduction

Hash functions are fundamental cryptographic primitives, essential for ensuring data integrity and supporting numerous security protocols [Mer89]. In recent years, permutation-based hash designs, particularly the sponge construction [BDPA08], have emerged as a predominant paradigm in lightweight cryptography because of their extreme simplicity, owing in part to the absence of a key schedule, and their high flexibility. This is evidenced by their adoption in the SHA-3 standard [Dwo15] and prominent lightweight algorithms such as Ascon [DEMS21]. A standard sponge construction updates its internal state by calling a b -bit public permutation, where the state is partitioned into a rate of r bits (dictating throughput) and a capacity of c bits (dictating security).

However, the standard sponge construction faces an inherent theoretical limitation: within the indistinguishability framework [MRH04], its generic security is strictly bounded by the birthday bound of the capacity, providing at most a security level of approximately $2^{c/2}$ [BDPA08]. In practical deployments, ensuring high security under the birthday bound often requires a harsh trade-off. Directly increasing the capacity c inevitably sacrifices the rate r , leading to a significant drop in throughput; conversely, enlarging the underlying permutation size b multiplicatively increases the hardware area and energy consumption.

To overcome this barrier, researchers have explored new hashing modes capable of achieving beyond-birthday-bound security. A prominent representative is the double sponge construction proposed in [LM24]. This scheme introduces the double-block-length

(DBL) paradigm [Hir06] into the permutation domain. By employing two parallel permutation calls, the double sponge construction raises the generic security level to $2^{2c/3}$. However, this security enhancement from parallel invocations introduces additional memory overhead: to maintain two independent branches, the total state space expands to $2r + 2c$ bits. In memory-sensitive lightweight applications, this increased state size can become a restrictive bottleneck for actual deployment. This motivates a natural research question in lightweight cryptography: Can we achieve equivalent beyond-birthday-bound security with a more compact state space, without enlarging the underlying permutation?

1.1 From Serial Block Ciphers to Permutations

In the domain of block-cipher-based hash designs, the double-block-length (DBL) architecture has been widely employed to construct high-security compression functions. To address the substantial memory overhead associated with traditional parallel DBL constructions, recent research, such as EXEX-NI [NSS21], has introduced a serial design philosophy. By invoking the underlying primitives serially and maximizing state reuse, this approach effectively minimizes the required memory footprint. This achievement naturally motivates a highly promising research direction: exploring the adaptation of this “serial space-saving” strategy into permutation-based hashing. The goal is to construct a new sponge framework that achieves both beyond-birthday-bound security and a minimal state space, strictly without enlarging the underlying primitive size.

However, translating this paradigm across different primitive domains faces a fundamental technical gap. In serial modes based on block ciphers (or tweakable block ciphers) like EXEX-NI, the cornerstone of their security relies heavily on the cryptographic isolation provided by the key. Even when successive encryption calls share a highly compressed intermediate state, the underlying key schedule inherently breaks the structural symmetry, prevents inner collisions, and ensures that each invocation provides independent and sufficient confusion and diffusion.

In contrast, the underlying permutation in the sponge construction is fully public and keyless. If two permutations are naively concatenated in series and forced to share the capacity state without a key, the two permutation calls will exhibit a high degree of structural symmetry due to the lack of domain separation. Stripped of the protection afforded by a key, such a straightforward serial compressed construction becomes vulnerable to inner collision attacks and fails to resist generic attacks exploiting this symmetry. Therefore, the core technical challenge of this work is as follows: in a public permutation environment, a fundamentally new state layout and feedback mechanism should be designed. This mechanism should enable the alternating serial invocation to not only successfully compress the state space but also intrinsically generate sufficiently strong diffusion to resist generic attacks against the visible state, thereby providing robust beyond-birthday-bound security guarantees within the indistinguishability framework.

1.2 Our Contribution: the Alternating Sponge

To bridge this gap, we propose a new hash function scheme named the **Alternating Sponge (ASP)**. The core intuition behind ASP is to replace the “spatial redundancy” of parallel paths with “temporal alternation” in a serial execution. To overcome the lack of key-based isolation in public permutations, ASP re-architects the state-update logic: it utilizes a two-phase serial permutation sequence to update a shared capacity. This temporal separation intrinsically breaks the structural symmetry, ensuring that the intermediate state is sufficiently diffused before the next absorption step. Consequently, ASP reduces the total state space from $2r + 2c$ bits down to $2r + c$ bits, saving c bits of memory, while preserving a beyond-birthday-bound security of $\min\{2^{\frac{2c}{3}}, 2^{\frac{r+c}{2}}\}$, which exceeds the standard capacity birthday bound.

Furthermore, we demonstrate that this serial dependency actually unlocks a higher squeezing efficiency. By incorporating a symmetric XOR sum of internal state blocks, ASP is capable of outputting $2r$ bits per round. This doubles the throughput compared to the double sponge construction, which is restricted to an r -bit output per round due to consistency constraints in its parallel simulation.

Our main contributions are summarized as follows:

- **A New Paradigm for Beyond-Birthday-Bound Secure Hashing:** We introduce the Alternating Sponge structure, a compact permutation-based design that achieves beyond-birthday-bound security ($2^{2c/3}$) within a $2r + c$ state. By effectively “recycling” the capacity across alternating permutation calls, ASP provides a new tradeoff among security, memory cost, and efficiency for resource-constrained environments.
- **High-Throughput Squeezing Mechanism:** We propose a new state-extraction logic for the squeezing phase. Unlike the parallel architecture of the double sponge construction, which is limited by consistency constraints in the simulator, our serial feedback design allows for $2r$ -bit output per round. Here, a squeezing round refers to one complete cycle consisting of the sequential application of both permutations f_1 and f_2 . Therefore, the $2r$ -bit output is obtained after both permutation calls are finished. This shows that moving from parallel to serial execution can simultaneously reduce memory overhead and improve data squeezing efficiency.
- **Indifferentiability Proof:** We present a security analysis within the indifferentiability framework. Our results show that, despite reducing the total state size, the scheme achieves a distinguishing advantage against generic attacks bounded by the dominant terms $O(\frac{q^3}{2^{2c}})$ (the capacity term) and $O(\frac{q^2}{2^{r+c}})$ (the primitive term), where q denotes the number of oracle queries. This confirms the theoretical soundness of our compact design, ensuring security comparable to the double sponge construction, exceeding the standard capacity birthday bound ($O(\frac{q^2}{2^c})$).

1.3 Comparison with Existing Architectures

1.3.1 Comparison with Sponge-Type Architectures

To evaluate the design tradeoffs of ASP, we compare it with representative sponge-type hashing architectures, including the standard sponge [BDPA07], the double sponge [LM24], and VFB [LLHL26]. The standard sponge offers a compact state and flexible rate-capacity choices, but its indifferentiability security is limited by the capacity birthday bound. The double sponge improves the security bound to the beyond-birthday-bound level, but this comes at the cost of maintaining two parallel sponge states, resulting in a memory size of $2r + 2c$.

VFB is another recent sponge-type construction whose indifferentiability bound approaches c . In the original VFB paper, the reported $2c$ state size refers to the formal internal or chaining state preserved between consecutive compression rounds. In this comparison, however, we follow the memory-size accounting used in low-memory hashing works such as EXEX-NI [NSS21] and the double sponge construction, where the working memory required during primitive calls is also counted. Under this convention, VFB requires the $(r + c)$ -bit permutation working state, the c -bit state word used after the permutation call, and the c -bit truncated message value used in the feed-forward computation, giving $r + 3c$ bits in total.

Under the same memory-size accounting, ASP maintains two r -bit rate branches and a single shared c -bit capacity branch. During each primitive call, one active rate branch is processed together with the shared capacity as an $(r + c)$ -bit permutation input, while

the other r -bit rate branch is retained for the next alternating update. The message block M_i and its permuted block $\sigma(M_i)$ can be absorbed at once before the corresponding permutation calls, so they do not need to be retained as additional feed-forward values after the primitive evaluation. Therefore, the memory size of ASP is $2r + c$ bits. This differs from the double sponge, which maintains two full $(r + c)$ -bit sponge states and hence requires $2r + 2c$ bits of memory.

Under this memory-size accounting, ASP provides a different tradeoff. It reduces the memory size of the double sponge while preserving beyond-birthday-bound security. Compared with VFB, ASP has a lower asymptotic security bound in the high-rate regime, but it does not rely on the restriction $r \geq c$ and does not require a distinct finalization or output-expander. Moreover, when $r < 2c$, ASP uses less working memory than VFB under the above implementation-level accounting. Thus, ASP is attractive in parameter regimes where memory efficiency, flexible rate-capacity choices, and a unified sponge-like iteration are prioritized.

Table 1: Memory-Size Comparison with Sponge-Type Architectures

Mode	Primitives per Round	Memory Size	Parameter Regime	Proven Security Bound	Squeezing Rate per Round	Reference
Sponge	1	$r + c$	flexible (r, c)	$\frac{c}{3}$	r	[BDPA07]
Double Sponge	2	$2r + 2c$	flexible (r, c)	$\frac{2c}{3}$	r	[LM24]
VFB	1	$r + 3c$ ¹	$r \geq c$	$V(c)$, approaching c	r	[LLHL26]
Alternating Sponge	2	$2r + c$	flexible (r, c)	$\min\{\frac{2c}{3}, \frac{r+c}{2}\}$	$2r$	Our Work

In Table 1, the squeezing rate is measured per mode-level round. For ASP, one such round consists of the serial application of both f_1 and f_2 in the squeezing phase, which produces two r -bit output blocks and hence gives a squeezing rate of $2r$ per round. In contrast, the double sponge also uses two primitive calls per round, but its parallel structure only releases one r -bit output block per round due to consistency constraints between the two sponge states. VFB also outputs r bits per output-expander call, while allowing multiple output blocks to be generated in parallel.

The output mechanisms of these constructions reflect different design choices. The standard sponge uses the usual sequential squeezing procedure, and the double sponge follows a parallel sponge-type squeezing structure. VFB adopts a distinct finalization procedure and an output expander, which is useful for parallel XOF generation in the high-rate regime. ASP instead keeps a unified sponge-like alternating iteration for both absorbing and squeezing, without requiring a separate output-expansion mechanism. Therefore, the main advantages of ASP are its reduced memory size compared with the double sponge, its $2r$ squeezing rate per mode-level round, and its more flexible rate-capacity choices compared with VFB.

As detailed in Section 1.2, our security bound $\kappa = \min\{\frac{2c}{3}, \frac{r+c}{2}\}$ is governed by two terms: the capacity term $O(q^3/2^{2c})$ and the primitive term $O(q^2/2^{r+c})$. The dominance of these terms depends on the threshold $r \approx c/3$:

- For $r \leq c/3$: The bound is limited by $O(q^2/2^{r+c})$. While slightly lower than the double-sponge bound, it still provides beyond-birthday-bound security.
- For $r > c/3$: The capacity term $O(q^3/2^{2c})$ dominates, matching the double-sponge bound.

Concrete Instantiation. Consider a 128-bit security target ($\kappa = 128$) using a $b = 320$ -bit primitive, such as an Ascon-style permutation:

¹The VFB paper reports a $2c$ -bit internal/chaining state. The $r + 3c$ value in this table follows the implementation-level memory-size accounting described above.

- **Sponge**: Requires $c = 256$, leaving $r = 64$. Memory size: 320 bits.
- **Double Sponge**: Requires $c = 192$, allowing $r = 128$. Its $2r + 2c$ structure requires 640 bits of memory.
- **VFB**: Since the security bound of VFB approaches c , one may take $c = 128$ and $r = 192$ under the same $b = 320$ bits primitive. VFB requires $r + 3c = 576$ bits of memory.
- **Alternating Sponge**: Requires $c = 192$ and $r = 128$, but its $2r + c$ structure requires only 448 bits of memory.

In this setting, ASP achieves the same security bound and absorption rate as the double sponge while reducing the memory size by 192 bits, i.e., about 30%. Compared with VFB, ASP uses less memory and preserves a more flexible rate-capacity tradeoff, whereas VFB offers a stronger high-rate security bound.

1.3.2 Comparison with Other Permutation-Based Hashing Modes

We further compare ASP with other prominent permutation-based modes, Grøstl and JH (Table 2), using the primitive size b as the baseline reference.

Table 2: Parameter Comparison with Other Permutation-Based Hashing Modes

Mode	Primitives per Round	Memory Size	Primitive Size	Absorbing Rate (per round)	Proven Bound (κ)	Best Known Attack (λ)	Reference
Grøstl	2	$3b$	b	b	$\frac{b}{4}$	$\frac{b}{2}$	[GKL ⁺ 11]
JH	2	b	b	$\frac{b}{2}$	$\frac{b}{4}$	$\frac{b}{2}$	[Wu11, LLHL26]
Alternating Sponge	2	$b + r$	b	r	$\min\{\frac{2c}{3}, \frac{r+c}{2}\}$	$\min\{\frac{2c+r}{3}, \frac{r+c}{2}\}$	Our Work

(Note: In the Alternating Sponge, $b = r + c$. Grøstl utilizes two independent permutations P and Q ; ASP similarly employs two independent permutations f_1 and f_2 .)

The architectural trade-offs are summarized as follows:

- **Grøstl (Wide-Pipe/Feed-Forward)**: Requires parallel permutations and buffering a b -bit chaining variable for its feed-forward operation. This inflates the hardware state to $3b$, creating a heavy memory overhead for a $b/4$ security bound.
- **JH (Security Bottleneck)**: While JH is state-efficient (state size = b), its security is strictly limited to $b/4$. To match ASPs security, JH would require a significantly larger, more computationally expensive primitive.
- **ASP (Optimized Balance)**: ASP is the only mode that maintains a compact b -bit primitive while breaking the memory-security bottleneck. By multiplexing two permutations (f_1, f_2) via “temporal alternation”, it limits the total state to $b + r$ while providing the beyond-birthday-bound security.

In summary, the comparison validates our core design philosophy: ASP is not merely a compromise for resource-constrained environments, but a flexible beyond-birthday-bound hashing design point.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 introduces the fundamental concepts and notations that will be used throughout the paper. Section 3 presents our new Alternating Sponge in detail, outlining its core architecture, state update mechanism, and the rationale behind its design choices. Section 4 is dedicated to the security analysis of the Alternating Sponge under the indistinguishability framework. Section 5 then provides

a detailed probability computation for bad events. Section 6 presents two attacks and discusses the tightness of the bound. Finally, Section 7 concludes the paper by summarizing our contributions and discussing potential avenues for future research.

2 Preliminaries

2.1 Notation

This subsection provides the fundamental notations and basic definitions used throughout this paper.

- $\mathbb{F}_2$: denotes the binary field $\{0, 1\}$.
- $r, c \in \mathbb{N}$: represent the rate and capacity parameters, respectively, in sponge construction.
- $b = r + c$: denotes the bit length on which each permutation function operates.
- f_1, f_2 : represent random permutation functions, where $f_i : \{0, 1\}^{r+c} \rightarrow \{0, 1\}^{r+c}$.
- σ : represents a linear message permutation, where $\sigma : \mathbb{F}_2^r \rightarrow \mathbb{F}_2^r$ is a linear function.
- $M \in \{0, 1\}^*$: denotes an arbitrary-length input message string, which can be divided into r -bit message blocks $M = M_1 || M_2 || \dots$.
- Z_i : denotes the i -th output segment.
- $(S_{up}, S_{mid}, C) \in \{0, 1\}^{2r+c}$: represents the current internal state, composed of an upper state segment $S_{up} \in \{0, 1\}^r$, a middle state segment $S_{mid} \in \{0, 1\}^r$, and a capacity segment $C \in \{0, 1\}^c$.
- $||$: denotes the bit string concatenation operation.
- $\oplus$: denotes the bitwise XOR operation.
- $\text{length}(M)$: denotes the bit length (in $\mathbb{N}$) of the bit string M .
- $X[a : b]$: denotes the bit substring of X starting at index a (inclusive) and ending at index b (inclusive).

2.2 Sponge and Double Sponge Constructions

This section provides a brief overview of the foundational sponge construction and its evolution into the double sponge construction, highlighting the core principles relevant to our low-memory Alternating Sponge.

2.2.1 The Sponge Construction

The sponge construction, proposed by Bertoni et al. [BDPA08], serves as a unified cryptographic primitive for hash functions and related applications. It operates through an **absorbing-squeezing** mechanism driven by an internal permutation f . The fixed internal state, of size b bits, is conceptually partitioned into a public rate portion (r bits) and a secret capacity portion (c bits), where the state size is defined as $b = r + c$.

In the **absorbing phase**, message blocks are XORed into the rate portion, followed by an application of f . During the **squeezing phase**, output blocks are extracted from the rate portion, with f applied between extractions. The construction's simplicity and strong provable security guarantees have led to its adoption in the SHA-3 standard.

2.2.2 The Double Sponge Construction

Building upon the sponge construction, the **double sponge construction** was proposed to enhance its security boundary. Its primary goal is to elevate the generic security level from the birthday bound of $2^{\frac{2c}{3}}$ to a higher level of $2^{\frac{2c}{3}}$. The main characteristic of this design is its use of two cooperating state units or processing paths, instead of a single one. This dual-path architecture enables more complex state management and stronger information diffusion.

Although the specific mixing functions and state update mechanisms may vary across different double sponge variants, the core principle involves parallel or alternating interactions between the two state units. However, this inherent design typically results in a **larger total state size** compared to that of a single sponge, posing challenges for deployment in memory-constrained environments.

2.3 RO-Consistency

RO-Consistency is a critical property for evaluating the reliability of a simulator's behavior within an ideal model. It requires that the simulator's output generation strictly aligns with that of a true random oracle.

We adopt the following definition for RO-consistency:

Definition 1. A simulator S for an iterated eXtendable-Output Function (XOF) construction is RO-consistent if for all $i \in \mathbb{N}$ and for all $M \in \{0, 1\}^*$, whenever one can compute $H_S(M)[0 : i]$ from the query history to the simulator, then it equals $RO(M)[0 : i]$.

Informally, this definition requires that the simulator S cannot “deceive” the adversary when generating outputs. An eXtendable-Output Function (XOF) is a cryptographic primitive that, unlike a traditional hash function, can produce an output of any desired length. Specifically, whenever the adversary is able to infer a prefix $H_S(M)[0 : i]$ of the hash value for a message M based on the simulators behavior, this prefix must be identical to what the random oracle RO would produce on input M . The value $H_S(M)$ is a theoretical hash value that the adversary can logically construct or infer from the results of the underlying primitive queries provided by the simulator. This ensures that the simulator's external behavior is indistinguishable from that of a genuine random oracle.

It is worth noting that RO-consistency can generally be guaranteed under certain conditions, for instance, **as long as a “bad event” does not occur**. In cryptographic proofs, a *bad event* refers to low-probability occurrences that would cause the simulator to malfunction or fail to perfectly mimic the ideal model such as an internal state collision, or the simulator being required to output contradictory results.

A successful security proof identifies all such bad events and demonstrates that their total probability is negligible, thereby deriving the overall security of the system.

3 The Alternating Sponge Construction

3.1 Algorithm Framework

3.1.1 State Partitioning and Permutation Process Design

Our construction employs an overall fixed-size state structure of $2r + c$ bits, which comprises three segments:

- **Upper State Segment** ($S_{up} \in \{0, 1\}^r$): Used for the first-stage permutation.
- **Middle State Segment** ($S_{mid} \in \{0, 1\}^r$): Used for the second-stage permutation.
- **Capacity Region** ($C \in \{0, 1\}^c$): Controls the security bound.

The permutation functions are denoted as f_1 and f_2 , which are used to process different state combinations and are called alternately during the absorbing phase to complete the state update.

The absorbing process for each message block M_i consists of two steps:

1. **First-Stage Permutation**

The input message block M_i is XORed with the upper state segment, and the result is concatenated with the current capacity region to form the input for permutation function f_1 :

$$(S'_{up}, C') = f_1(S_{up} \oplus M_i || C).$$

2. **Second-Stage Permutation**

The permuted message block $\sigma(M_i)$ is XORed with the middle state segment. Here, σ is a **linear permutation function**. It is included primarily for structural flexibility and tunability, as it allows for differentiation between the message inputs of the two stages at a negligible cost. Our security proof does not impose any additional conditions on σ ; for a concrete implementation seeking maximum efficiency, σ can simply be set to the identity function.

This result is then concatenated with the capacity region C' (updated from the first stage) and input to permutation function f_2 :

$$(S'_{mid}, C'') = f_2(S_{mid} \oplus \sigma(M_i) || C').$$

3. **State Update**

The overall state is updated in this round as:

$$(S_{up}, S_{mid}, C) \leftarrow (S'_{up}, S'_{mid}, C'').$$

This design ensures the compactness and diffusion of each round's processing through layered absorption and alternating permutations. Simultaneously, it controls the total state size to be within $2r + c$ bits, which is significantly less than the total state size of $2r + 2c$ in the original double sponge construction, while maintaining the same permutation function size of $r + c$ bits.

3.1.2 Squeezing Phase and Output Mechanism

The squeezing phase is employed after the absorbing phase to progressively extract the output digest from the current state. This phase reuses permutation functions f_1 and f_2 , continuously applying them to different parts of the current state to generate the desired length output bit string $Z = z_0 || z_1 || z_2 || \dots$.

Similarly, the state structure remains in three parts:

- Upper State Segment ($S_{up} \in \{0, 1\}^r$);
- Middle State Segment ($S_{mid} \in \{0, 1\}^r$);
- Capacity Region ($C \in \{0, 1\}^c$).

In the squeezing phase, the output process adopts a two-stage alternating fragment generation method. Each round produces two output blocks while simultaneously updating the internal state. Assuming the current state is $(S_{up}, S_{mid}, C) \in \{0, 1\}^r \times \{0, 1\}^r \times \{0, 1\}^c$, and the input length for both permutation functions f_1, f_2 is $r + c$ bits, then the output process for each round is as follows:

1. **Initial Output (z_0):**

Prior to the first squeezing permutation, the initial fragment z_0 is extracted from the final absorbing states:

$$z_0 = S_{up} \oplus S_{mid}$$

2. **Iterative Generation Process (Round $i \geq 0$)**

Each round generates a pair of fragments through the following unified stages:

• **Stage 1: f_1 Permutation**

The current even-indexed fragment z_{2i} (with z_0 as the base case for $i = 0$) serves as the rate-part input for permutation f_1 :

$$(S'_{up}, C') = f_1(z_{2i} \parallel C)$$

• **Stage 2: Extraction of z_{2i+1} and f_2 Permutation**

The odd fragment z_{2i+1} is produced by XORing the intermediate output S'_{up} with the middle state S_{mid} ; this result then serves as the rate input for the second permutation f_2 :

$$z_{2i+1} = S'_{up} \oplus S_{mid}$$

$$(S'_{mid}, C'') = f_2(z_{2i+1} \parallel C')$$

• **Stage 3: Extraction of z_{2i+2} and State Update**

The next even fragment z_{2i+2} is derived from the XOR sum of the rate outputs from both permutations; this value then updates the internal state to finalize the current round and initialize the next:

$$z_{2i+2} = S'_{up} \oplus S'_{mid}$$

$$(S_{up}, S_{mid}, C) \leftarrow (z_{2i+2}, S'_{mid}, C'')$$

This mechanism, by alternately using two permutations and state XOR operations during the output process, enhances state interleaving and diffusion capabilities, allowing for the iterative generation of output digests of arbitrary length.

By continuously alternating calls to f_1 and f_2 and extracting outputs, the squeezing phase can flexibly produce digest values of any length while maintaining robust diffusion and security through serial path dependency. The output rate is $2r$ bits per round, which is twice the r -bit output rate of the double sponge construction, but the total state size is compressed to $2r + c$ bits, improving overall memory efficiency.

To clearly illustrate the state partitioning and permutation flow of the Alternating Sponge, Figure 1 presents its overall architecture. Dashed lines indicate state transfers rather than direct inputs to the current-round permutation. A more formal description is provided in the pseudocode (Algorithm 1), where the absorbing and squeezing phases are presented together in a single main procedure for a step-by-step overview of the process.

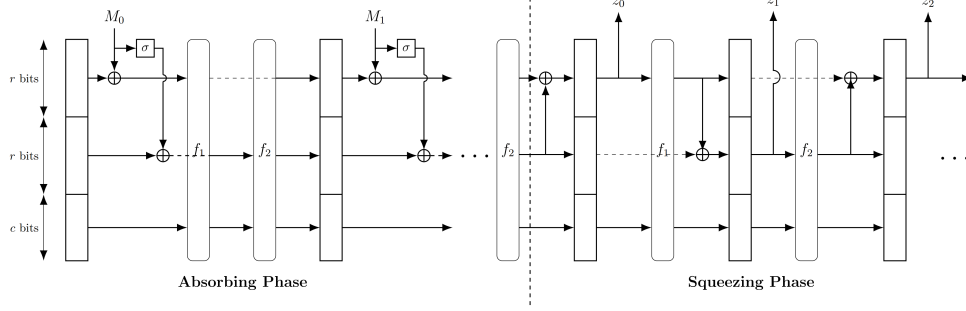


Figure 1: Structure diagram of ASP

Algorithm 1 Alternating Sponge Operation Process

Require: Message M , desired output length l , rate r , capacity c

Ensure: Hash digest D

```

1: // 1. Padding
2:  $M' \leftarrow \text{Pad}(M, r)$ 
3: // 2. State Initialization
4:  $S_{up} \leftarrow 0^r; S_{mid} \leftarrow 0^r; C \leftarrow 0^c$ 
5: // 3. Absorbing Phase
6:  $\text{num\_absorb\_blocks} \leftarrow \text{length}(M')/r$ 
7: for  $i$  from 0 to  $\text{num\_absorb\_blocks} - 1$  do
8:    $M_i \leftarrow M'[(i \cdot r) : ((i + 1) \cdot r - 1)]$ 
9:    $(S'_{up}, C') \leftarrow f_1(S_{up} \oplus M_i \parallel C)$ 
10:   $(S'_{mid}, C'') \leftarrow f_2(S_{mid} \oplus \sigma(M_i) \parallel C')$ 
11:  // State Update
12:   $S_{up} \leftarrow S'_{up}; S_{mid} \leftarrow S'_{mid}; C \leftarrow C''$ 
13: end for
14: // 4. Squeezing Phase
15:  $\text{output\_digest} \leftarrow \text{empty string}$ 
16:  $S_{up} \leftarrow S_{up} \oplus S_{mid}$  // Initial XOR blending ( $z_0$ )
17: while  $\text{length}(\text{output\_digest}) < l$  do
18:    $z_{2i} \leftarrow S_{up}$  // Current even fragment
19:    $(S'_{up}, C') \leftarrow f_1(z_{2i} \parallel C)$ 
20:    $z_{2i+1} \leftarrow S'_{up} \oplus S_{mid}$  // Extraction of odd fragment
21:    $(S'_{mid}, C'') \leftarrow f_2(z_{2i+1} \parallel C')$ 
22:   // State Update and Next Even Fragment ( $z_{2i+2}$ )
23:    $S_{up} \leftarrow S'_{up} \oplus S'_{mid}$ 
24:    $S_{mid} \leftarrow S'_{mid}; C \leftarrow C''$ 
25:   // Append fragments to the digest
26:    $\text{output\_digest} \leftarrow \text{output\_digest} \parallel z_{2i} \parallel z_{2i+1}$ 
27: end while
28: // 5. Return the result truncated to  $l$  bits
29: return  $\text{output\_digest}[0 : l - 1]$ 

```

Algorithm 2 Pad function (10* rule)

```

1: function Pad( $M, r$ )
2: // Pads the message M using the 10* rule
3:  $M' \leftarrow M \parallel '1'$ 
4: while  $\text{length}(M') \pmod{r} \neq 0$  do
5:    $M' \leftarrow M' \parallel '0'$ 
6: end while
7: return  $M'$ 
8: end function

```

3.2 Message Padding Mechanism

To ensure input messages can be divided into r -bit blocks and to prevent ambiguities arising from all-zero trailing blocks, our construction adopts a padding strategy consistent with that used in the original sponge and double sponge constructions. This strategy mandates that the padded message length must be an integer multiple of r , and the final block must not consist entirely of zeros. This ensures both the uniqueness and security of the block division.

Specifically, we utilize the **10* padding rule**, which operates as follows:

1. A single '1' bit is appended to the end of the original message M .
2. This is followed by appending the minimum number of '0' bits required.
3. The goal is for the total length of the padded message M' to be an exact multiple of r .

The function $\text{Pad}(M, r)$ described in Algorithm 2 formally implements this padding scheme. This padding function is **injective**, meaning that distinct messages will always produce distinct padded encodings. This avoids padding conflicts that could lead to digest collisions, which is a fundamental requirement for hash function security, ensuring a unique representation for each message and, consequently, a unique final digest.

Since our construction's absorbing process still operates on r -bit units per round, this padding method directly integrates with our proposed two-stage absorption flow without requiring any additional modifications.

3.3 Practical Considerations: Single vs. Dual Permutations

Our security proof in Section 4, for clarity and rigor, assumes that f_1 and f_2 are two independent, random permutations. However, in a practical (especially hardware) implementation, instantiating two different large permutations would double the resource cost (e.g., circuit area). This would largely negate the state memory advantage ($2r + c$ bits) that ASP provides.

This is a common consideration for multi-permutation constructions. In practice, as noted in the double sponge construction approach [LM24], f_1 and f_2 can be instantiated from a single b -bit permutation $\mathcal{P}$ using **internal domain separation**. This is achieved by logically fixing the first bit of the input for separation purposes:

$$f_1(X) := \mathcal{P}(0 \parallel \cdot) \quad f_2(X) := \mathcal{P}(1 \parallel \cdot)$$

This technique ensures that f_1 and f_2 behave as two distinct functions from a cryptographic standpoint, thus preserving the structure of our security proof, while only requiring a single permutation (and a negligible XOR cost) in the final implementation.

4 Security Analysis

In this section, we provide a formal proof of the security bound. We demonstrate that our Alternating Sponge is indistinguishable from a random oracle ($\mathcal{RO}$) with a distinguishing advantage bounded by $\mathcal{O}(\frac{q^3}{2^{2\epsilon}})$ and $\mathcal{O}(\frac{q^2}{2^{r+\epsilon}})$. This result, which confirms the claim in Theorem 1, shows that our scheme achieves a level of security comparable to that of the double sponge construction.

4.1 Indifferentiability Security Model

This subsection formally defines the indifferentiability security concept that we adopt, and clarifies the adversary model, laying the foundation for the subsequent security proof.

4.1.1 Definition of Indifferentiability

The indifferentiability framework, proposed by Maurer et al. in 2004 [MRH04], is a powerful security concept in cryptography used to assess whether a cryptographic construction achieves equivalent security under idealized primitives (such as a random oracle). It goes beyond traditional properties like collision resistance or preimage resistance, aiming to prove the computational indistinguishability of the construction as a whole from an ideal primitive. This framework was further extended and refined in the context of hash functions by Coron et al. [CDMP05], providing more detailed techniques for analyzing complex constructions.

Definition 2 (Indifferentiability). Let $\mathcal{F} : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be an ideal random oracle (or another ideal primitive), and $\mathcal{C}$ be a hash function construction built using underlying concrete primitives $\mathcal{P}$ (e.g., b -bit permutations P_1, P_2). We say that the construction $\mathcal{C}$ is computationally indifferentiable from $\mathcal{F}$ under an accessed oracle family $\mathcal{O}$, if for any Probabilistic Polynomial-Time (PPT) distinguisher $\mathcal{A}$, there exists a Probabilistic Polynomial-Time (PPT) simulator $\mathcal{S}$ such that the outputs of the following two experiments are computationally indistinguishable:

1. **Real World (Exp_{Real}):** The distinguisher $\mathcal{A}$ has oracle access to the actual construction $\mathcal{C}$ and its underlying concrete primitives $\mathcal{P}$. That is, $\mathcal{A}$ interacts with $(\mathcal{C}, \mathcal{P})$.
2. **Ideal World ($\text{Exp}_{\text{Ideal}}$):** The distinguisher $\mathcal{A}$ has oracle access to the simulated pair $(\mathcal{F}, \mathcal{S}^{\mathcal{F}})$. $\mathcal{F}$ represents the random oracle ($\mathcal{RO}$), and $\mathcal{S}^{\mathcal{F}}$ represents the simulated underlying primitives. The simulator $\mathcal{S}$ acts as an agent, responsible for answering primitive queries by making calls to the ideal primitive $\mathcal{F}$.

The advantage of the distinguisher $\mathcal{A}$ is defined as:

$$\text{Adv}_{\mathcal{C}, \mathcal{F}}^{\text{indiff}}(\mathcal{A}) = \left| \Pr[\mathcal{A}^{(\mathcal{C}, \mathcal{P})} = 1] - \Pr[\mathcal{A}^{(\mathcal{F}, \mathcal{S}^{\mathcal{F}})} = 1] \right|.$$

If for any PPT distinguisher $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{C}, \mathcal{F}}^{\text{indiff}}(\mathcal{A})$ is a negligible function of the security parameter κ , then we say that the construction $\mathcal{C}$ is indifferentiable from the ideal primitive $\mathcal{F}$.

Intuitively, indifferentiability implies that no attacker can, through any polynomial-time query behavior, distinguish whether it is facing the actual construction and its real underlying primitives, or an ideal black-box and its underlying behavior simulated by a simulator. If this property holds, then any protocol proven secure under the ideal model will retain its security when the ideal primitive is replaced by this construction.

4.1.2 Adversary Model

In this proof, we employ the indistinguishability analysis framework to evaluate the security of our construction. The adversary $\mathcal{A}$ we consider is a Probabilistic Polynomial-Time (PPT) algorithm [Gol01], whose ultimate goal is to succeed with a probability significantly better than a random guess at determining its environment. If the adversary can successfully guess the room it's in with a non-negligible probability, it implies there's an exploitable difference between our construction and an ideal hash function.

The total number of queries q issued by the adversary $\mathcal{A}$ throughout the experiment is polynomially bounded (i.e., $q = \text{poly}(\kappa)$). We typically further differentiate query limits for various oracles, such as q_c queries to the hash function, q_{f_1} queries to f_1 and q_{f_2} queries to f_2 .

The ultimate goal of the adversary $\mathcal{A}$ is to successfully distinguish, with a non-negligible probability, which environment it is in, without knowing. If it believes it is in the real world, it outputs 1; otherwise, it outputs 0. Our proof aims to show that no such PPT adversary can achieve this with a significant advantage.

Theorem 1. *Let $\mathcal{H}$ be our proposed hash function construction, which uses the real permutations f_1 and f_2 . Let $\mathcal{S}$ be the simulator in the ideal model with a random oracle $\mathcal{RO}$. For any probabilistic polynomial-time (PPT) distinguisher $\mathcal{D}$ that makes a total of at most q queries (including queries to the hash function itself and to the underlying permutations f_1, f_2), the indistinguishability advantage of our construction is bounded by:*

$$\text{Adv}_{\mathcal{H}, \mathcal{S}}^{\text{indif}}(\mathcal{D}) \leq \frac{216q^3}{(2^c - 4q)^2} + \frac{6q^{3/2}}{2^c - 4q} + \frac{q^2 + q}{2^{2r+c+1}} + \frac{4q^2 - 4q}{2^{r+c}},$$

where c is the size of the capacity region and q represents the total number of queries made by the distinguisher.

We will prove the security bound stated in Theorem 1 through a step-by-step analysis. Our proof is based on the world decomposition method, where we quantify the gap between the ideal world and the real world. This process involves formally defining the simulator (4.2), identifying potential bad events (4.3), and finally, applying the world decomposition (4.4) to derive the final bound. The specific probability calculations and the derivation of the bounds for each term will be detailed in the following sections.

4.2 Simulator Construction

4.2.1 State Definition and Graph Structure

To formalize the security analysis of the ASP construction, the simulator maintains a dynamic directed labeled graph $G = (V, \mathcal{E})$ that records its knowledge of the internal state space.

In this model, the internal state of ASP is abstracted as a triplet $N = (A, B, C) \in \{0, 1\}^{2r+c}$, where A and B denote the r -bit upper and middle state components, respectively, and C represents the c -bit capacity component. The node set V is a dynamic subset of the full state space, initially containing only the initial vector $IV = (IV_A, IV_B, IV_C)$.

Edges in G represent absorption transitions between states. An edge $u \xrightarrow{m} v$, labeled by a message block $m \in \{0, 1\}^r$, exists if and only if, for a starting node $u = (A, B, C)$ and message block m , there exist entries in query tables $table_{f_1}$ and $table_{f_2}$ that constitute a complete absorption step.

The query tables themselves only store $(r+c)$ -bit primitive input-output pairs (X, Y) . The simulator interprets these raw bitstrings as rate portions $Y.\text{rate}$ and capacity portions $Y.\text{cap}$, and verifies whether they satisfy the structural constraints imposed by the ASP design. Once the construction rules are satisfied, the internal state of the target node v

is uniquely determined by the rate and capacity components of these two permutation outputs. A state $(A, B, C) \in V$ is called a **rooted node** if it is reachable from the initial state (IV_A, IV_B, IV_C) through a finite number of absorption steps. The set of all such nodes is denoted as *Rooted*.

To ensure a consistent and complete view of the state space, the simulator maintains G such that it is always **maximal** with respect to the query history. This property implies that any path logically implied by the entries in the query tables and reachable from IV must be explicitly contained in $\mathcal{E}$. The specific constraints and the mechanism used to uphold this maximality are detailed in Section 4.2.2.

4.2.2 Simulator's Internal Structure and Consistency Constraints

The simulator $\mathcal{S}$ maintains the following core data structures, which collectively record the adversary's query history and preserve the internal states consistency:

- **Query Tables ($\text{table}_{f_1}, \text{table}_{f_2}$):** The simulator utilizes two hash tables to record the entire query history of the permutations f_1 and f_2 . Each table maps an $(r + c)$ -bit raw input bitstring to an $(r + c)$ -bit output bitstring. To support bidirectional lookups, each query is stored as a pair of entries: a forward entry (mapping input X to output Y) and an inverse entry (mapping output Y to input X).
- **State Transition Graph $\mathcal{G}$:** A conceptual directed graph $G = (V, \mathcal{E})$ representing state transitions during the absorption phase. Its edge structure is dynamically reconstructed from the raw entries in the query tables.

From the simulator's perspective, entries in the aforementioned tables are strictly classified into two categories:

- **Integrated Entries:** These refer to entries that have been incorporated as edges into the graph G and belong to a deterministic path originating from the initial state IV .
- **Wild Entries:** These entries exist in the query records but have not yet been identified as part of any rooted path. The simulator stores only the raw bitstrings for these entries and does not presuppose any specific absorption structure (such as $A \oplus m$) until they are triggered and closed by subsequent logic.

Within the simulator's operational logic, the evolution of an entry from a "wild state" to a "graph edge" is fundamentally driven by a structural signal known as a "partial extension".

Definition 3 (Partial Extension). A partial extension describes an intermediate state where a connection between a rooted node and a table entry has been established but not yet resolved into a complete edge $u \xrightarrow{m} v$. Based on the serial dependency of the ASP construction, partial extensions are categorized as follows:

- **Forward Partial Extension:** A rooted node $u \in \text{Rooted}$ and a message m exist such that the corresponding f_1 entry is present in table_{f_1} , but the subsequent f_2 entry is missing.
- **Backward Partial Extension:** A rooted node $u \in \text{Rooted}$ and an entry in table_{f_2} exist such that the input capacity portion of the f_2 entry matches the output capacity portion of a potential f_1 call from u , yet the connection remains unclosed.

The simulator enforces a **Structural Integrity Constraint**. Whenever a wild entry satisfies the matching conditions of a partial extension due to the addition of a new query, the simulator must immediately initiate a completion procedure.

Specifically, the simulator recursively scans all newly formed connections via the `UpdateGraph()` process. For every detected partial extension, the simulator defines the missing permutation values either through lazy sampling or by solving algebraic consistency equations depending on whether the current path is constrained by the Random Oracle ($\mathcal{RO}$). This procedure ensures that f_1 and f_2 always occur in pairs along any rooted path, thereby maintaining the graph G in a maximal state with respect to the current query tables.

Such partial structures represent critical points of uncertainty within the simulation. If these connections remain unresolved, or if their mandatory completion enforced by the `UpdateGraph()` process conflicts with existing table entries, it leads to logical contradictions. In the security proof, these contradictions are categorized as **bad events**. The occurrence of a bad event indicates that the simulator can no longer perfectly replicate the real construction, potentially enabling the adversary to distinguish the simulated environment from the real one.

By enforcing the maximality of G , the simulator proactively identifies and resolves such structural risks. This ensures that all reachable paths originating from IV are consistently and uniquely defined within the graph, preventing the adversary from distinguishing the simulated environment by probing for hanging inconsistencies in the graph structure.

4.2.3 Simulator's Query Processing Algorithm

Upon receiving a query (either forward or inverse) to the underlying permutations f_1 or f_2 , the simulator $\mathcal{S}$ executes a four-phase processing procedure. This procedure is designed to dynamically construct the state transition graph G while ensuring that the simulator's responses remain logically indistinguishable from those of the Random Oracle $\mathcal{RO}$ and that the inherent randomness of the permutations is preserved. The overall logic of the query handling is summarized in Algorithm 3, which orchestrates the transition from context identification to recursive graph updates.

Algorithm 3 Main Simulator Procedure

Require: Permutation index $i \in \{1, 2\}$, Direction $dir \in \{fwd, inv\}$, Query value W

Ensure: Response Y

```

1: // 1. Context Identification
2:  $(\mathcal{C}_{active}, X) \leftarrow \text{Algorithm 4}(i, dir, W)$ 
3: // 2. Resolution and Assignment Flow
4: if  $\mathcal{C}_{active} \neq \emptyset$  then
5:   // Coherent Query: Resolve consistency before assignment
6:    $Y_{target} \leftarrow \text{Algorithm 5}(\mathcal{C}_{active}, i, X)$ 
7: else
8:   // Wild Query: Proceed directly to assignment with no constraint
9:    $Y_{target} \leftarrow \perp$ 
10: end if
11: // 3. Execution of Value Assignment
12:  $V_{old} \leftarrow \text{Rooted}$  // Store the state of Rooted nodes prior to update
13:  $Y \leftarrow \text{Algorithm 6}(i, X, Y_{target}, \mathcal{C}_{active})$ 
14: // 4. Recursive Graph Update
15: // Identify nodes newly added to the Rooted set during assignment
16: for each  $v \in (\text{Rooted} \setminus V_{old})$  do
17:   Algorithm 7( $v$ )
18: end for
19: return  $Y$ 

```

In the following, we provide a detailed technical exposition of each phase as outlined

in the procedure above.

1. Context Identification. In the initial phase, the simulator determines whether the current query input “bridges” a gap to any existing rooted path. It scans the *Rooted* set and identifies all active candidate paths affected by the query, aggregating them into the set $\mathcal{C}_{active}$. Formally, each candidate path is represented as a tuple (N, m) , where N is a predecessor rooted node and m is the message block corresponding to the current transition.

- **For f_1 Forward Queries:** The simulator checks whether the input capacity matches the capacity component of any rooted node N . If a match is found, it recovers the implicit message $m = R_{in} \oplus N.S_{up}$ and adds the path expansion (N, m) to $\mathcal{C}_{active}$.
- **For f_2 Forward Queries:** Due to the serial structure of ASP, the simulator performs a backward lookup in $table_{f_1}$ to identify potential predecessor entries. If an entry is found whose output capacity matches C_{in} and whose input is associated with a rooted node N , the simulator recovers the message m and verifies the rate consistency: $R_{in} \stackrel{?}{=} N.S_{mid} \oplus \sigma(m)$. Validated path expansions (N, m) are added to $\mathcal{C}_{active}$.
- **For Inverse Queries:** The simulator employs a “sample-then-verify” strategy. It first samples a value X to serve as the input (or primitive preimage) such that $f(X) = Y$. If the randomly sampled X unexpectedly bridges an existing path fragment to the *Rooted* set, the query transitions from “wild” to “coherent”, immediately triggering the subsequent resolution and update procedures.

If $\mathcal{C}_{active}$ is empty, the query is categorized as a *wild query*; otherwise, it is a *coherent query*. Detailed matching logic is provided in Algorithm 4.

2. Consistency Resolution. For coherent queries, the simulator first determines whether the output is constrained by $\mathcal{RO}$. By traversing G , it reconstructs the complete message history M_{full} for each candidate path in $\mathcal{C}_{active}$.

- **Intermediate Absorbing Phase:** For transitions within the absorption phase that occur prior to the f_2 call associated with the final message block, the output is treated as *free*. In this case, the simulator generates responses using standard lazy sampling.
- **Constrained Phase:** The constrained phase begins at the f_2 call corresponding to the final message block (which determines the first hash digest z_0) and extends to all subsequent squeeze calls. During this phase, the simulator must solve for a unique target value Y_{target} according to the relation $Z = S'_{up} \oplus S'_{mid}$ to ensure consistency with the Random Oracle output $\mathcal{RO}(M_{full})$.

The simulator allows *double-path collisions*, where two distinct paths M_1 and M_2 lead to the identical query input X . However, if the active candidate set becomes too large (i.e., $|\mathcal{C}_{active}| > 2$), a **3Col** bad event is triggered.

Additionally, if multiple candidate paths induce mutually inconsistent output-rate constraints on the same query, a conflict event is triggered. The formal resolution process is detailed in Algorithm 5.

Algorithm 4 Context Identification**Require:** Permutation index i , Query direction dir , Query value W **Ensure:** Candidate path set $\mathcal{C}_{active}$, Finalized input X

```

1: // 1. Determine Input  $X$ 
2: if  $dir = inv$  then
3:   // The query value  $W$  is the output  $Y$ 
4:   if  $\exists(\cdot, W) \in table_{fi}$  then
5:      $X \leftarrow table_{fi}^{inv}[W]$  // Retrieve  $X$  from inverse mapping
6:   else
7:      $X \xleftarrow{\$} \{0, 1\}^{r+c} \setminus \{X' \mid (X', \cdot) \in table_{fi}\}$ 
8:     // Note: We don't update the table until Algorithm 6
9:   end if
10: else
11:   // The query value  $W$  is the input  $X$ 
12:    $X \leftarrow W$ 
13: end if
14: // 2. Path Matching
15:  $R_{in} \parallel C_{in} \leftarrow X$ 
16:  $\mathcal{C}_{active} \leftarrow \emptyset$ 
17: if  $i = 1$  then
18:   for each node  $N$  in  $Rooted$  do
19:     if  $N.C = C_{in}$  then
20:        $m \leftarrow R_{in} \oplus N.S_{up}$ 
21:        $\mathcal{C}_{active} \leftarrow \mathcal{C}_{active} \cup \{(N, m)\}$ 
22:     end if
23:   end for
24: else
25:    $\mathcal{L} \leftarrow \{Entry \in table_{f1} \mid Entry.Y.cap = C_{in}\}$ 
26:   for each  $Entry$  in  $\mathcal{L}$  do
27:     if  $\exists N \in Rooted$  such that  $N.C = Entry.X.cap$  then
28:        $m \leftarrow Rate(Entry.X) \oplus N.S_{up}$ 
29:       if  $R_{in} = N.S_{mid} \oplus \sigma(m)$  then
30:          $\mathcal{C}_{active} \leftarrow \mathcal{C}_{active} \cup \{(N, m)\}$ 
31:       end if
32:     end if
33:   end for
34: end if
35: // 3. Decision
36: if  $\mathcal{C}_{active} \neq \emptyset$  then
37:   Go to Algorithm 4 // Coherent Query
38: else
39:   Go to Algorithm 5 // Wild Query
40: end if
41: return  $\mathcal{C}_{active}, X$ 

```

Algorithm 5 Consistency Resolution**Require:** Candidate set $\mathcal{C}_{active}$, Permutation index $i \in \{1, 2\}$, Query input X **Ensure:** Target output Y_{target}

```

1: // 1. Multi-path Collision Check
2: if  $|\mathcal{C}_{active}| > 2$  then
3:   Trigger 3Col and terminate
4: end if
5:  $Y_{target} \leftarrow \perp$ 
6: // 2. Constraint Derivation for Coherent Paths
7: for each  $(N, m) \in \mathcal{C}_{active}$  do
8:    $M \leftarrow N.M_{history} \parallel m$ 
9:   if  $M$  is in Constrained Phase then
10:    if  $i = 1$  then
11:       $R_{path} \leftarrow \mathcal{RO}(M)[2j+1] \oplus N.S_{mid}$  // Solve for  $z_{2j+1}$ 
12:    else
13:      // Retrieve the  $r$ -bit rate part from the precursor  $f_1$  output
14:       $S'_{up} \leftarrow \text{Rate}(\text{table}_{f_1}[N.S_{up} \oplus m \parallel N.C])$ 
15:      if  $M$  is the final absorbing block then
16:         $idx \leftarrow 0$  // Target the initial fragment  $z_0$ 
17:      else
18:         $idx \leftarrow 2j+2$  // Target the next even fragment  $z_{2j+2}$ 
19:      end if
20:       $R_{path} \leftarrow \mathcal{RO}(M)[idx] \oplus S'_{up}$  // Satisfy the ASP output equation
21:    end if
22:     $C_{rand} \xleftarrow{\$} \{0, 1\}^c$ 
23:     $Y_{path} \leftarrow R_{path} \parallel C_{rand}$ 
24:    // 3. Inconsistency Detection
25:    if  $Y_{target} \neq \perp$  and  $\text{Rate}(Y_{target}) \neq R_{path}$  then
26:      Trigger Inconsistency and terminate
27:    end if
28:     $Y_{target} \leftarrow Y_{path}$ 
29:  end if
30: end for
31: return  $Y_{target}$ 

```

3. Value Assignment. Once the constraint status has been determined, the simulator proceeds to execute the corresponding value-assignment logic.

- **Fresh Query Assignment:** For new inputs in a free state, the simulator performs standard lazy sampling. In a constrained state, the simulator assigns the target value Y_{target} , by fixing the rate portion to exactly satisfy the $\mathcal{RO}$ constraint, while sampling the capacity portion uniformly at random to preserve entropy.
- **Existing Entry Verification:** If the query input already exists in the tables, the simulator verifies that the stored value Y_{stored} is compatible with the current consistency constraints. Any irreconcilable mismatch between the previously sampled value and the subsequently imposed $\mathcal{RO}$ constraint results in an **Inconsistency** bad event, denoted $\mathbf{BAD}_{inconsist}$.

Once assignment is successfully completed without a bad event, the query is formally closed with its predecessor path. This process is formalized in Algorithm 6.

4. Recursive Graph Update. As the final phase, the simulator invokes the `UpdateGraph()` procedure to integrate the newly verified transition into G and update

Algorithm 6 Value Assignment and Edge Completion

Require: Index i , Input X , Target Y_{target} , Candidate set $\mathcal{C}_{active}$ **Ensure:** Final response Y

```

1: // 1. Verification of Existing Entries
2: if  $X \in \text{Dom}(\text{table}_{fi})$  then
3:    $Y_{stored} \leftarrow \text{table}_{fi}[X]$ 
4:   if  $Y_{target} \neq \perp$  and  $\text{Rate}(Y_{stored}) \neq \text{Rate}(Y_{target})$  then
5:     Trigger Inconsistency and terminate
6:   end if
7:    $Y \leftarrow Y_{stored}$ 
8:   // 2. Assignment for Fresh Queries
9: else
10:  if  $Y_{target} \neq \perp$  then
11:     $C_{rand} \xleftarrow{\$} \{0,1\}^c$ 
12:     $Y \leftarrow \text{Rate}(Y_{target}) \parallel C_{rand}$  // Satisfy  $\mathcal{RO}$  constraint
13:  else
14:     $Y \xleftarrow{\$} \{0,1\}^{r+c}$  // Standard lazy sampling
15:  end if
16:   $\text{table}_{fi}[X] \leftarrow Y$ 
17: end if
18: // 3. Completion of Partial Extensions
19: for each  $(N, m) \in \mathcal{C}_{active}$  do
20:   // Retrieve fixed results for this path from tables
21:    $(S'_{up}, C') \leftarrow \text{table}_{f1}[N.S_{up} \oplus m \parallel N.C]$ 
22:    $(S'_{mid}, C'') \leftarrow \text{table}_{f2}[N.S_{mid} \oplus \sigma(m) \parallel C']$ 
23:   // Define next state based on ASP update rule
24:    $v \leftarrow (S'_{up} \oplus S'_{mid}, S'_{mid}, C'')$ 
25:   if  $v \in \text{Rooted}$  then
26:     Trigger BadNode and terminate
27:   end if
28:    $\mathcal{E} \leftarrow \mathcal{E} \cup \{N \xrightarrow{m} v\}$ ;  $\text{Rooted} \leftarrow \text{Rooted} \cup \{v\}$ 
29: end for
30: return  $Y$ 

```

the *Rooted* set. Since the addition of a new node may retroactively activate previously wild table entries, the simulator recursively scans for and resolves all newly formed Partial Extensions.

This recursive completion mechanism ensures that the graph remains maximal. To guarantee termination, the recursion depth is strictly bounded by the **ThreeRounds** bad event. The recursive update procedure and its termination conditions are formalized in Algorithm 7.

Algorithm 7 Recursive Graph Update Procedure

```

1: Procedure UpdateGraph( $v_{start}$ )
2:  $Q \leftarrow \text{Queue}\{v_{start}\}$ 
3:  $depth(v_{start}) \leftarrow 1$ 
4: while  $Q$  is not empty do
5:    $u \leftarrow Q.\text{dequeue}()$ 
6:   // 1. Scan tables for newly reachable transitions
7:    $\mathcal{S}_{match} \leftarrow \{m \in \{0, 1\}^r \mid \text{path}(u, m) \text{ is fully defined in } table_{f_1}, table_{f_2}\}$ 
8:   for each  $m \in \mathcal{S}_{match}$  do
9:     if edge  $u \xrightarrow{m} \cdot$  is not in  $\mathcal{E}$  then
10:      // 2. Retroactive Consistency and Assignment
11:       $\mathcal{C}_{active} \leftarrow \{(u, m)\}$ 
12:       $Y_{target} \leftarrow \text{Algorithm 4}(\mathcal{C}_{active}, \dots)$ 
13:       $Y \leftarrow \text{Algorithm 5}(X, Y_{target}, \mathcal{C}_{active})$ 
14:      // 3. Recursive depth control
15:       $v_{new} \leftarrow$  the node resulting from edge  $(u, m)$ 
16:       $depth(v_{new}) \leftarrow depth(u) + 1$ 
17:      if  $depth(v_{new}) > 2$  then
18:        Trigger ThreeRounds and terminate
19:      end if
20:       $Q.\text{enqueue}(v_{new})$ 
21:    end if
22:  end for
23: end while
24: End Procedure

```

4.3 Bad Events Analysis

In the context of proving indistinguishability, the core task lies in constructing a simulator $\mathcal{S}$ whose behavior is computationally indistinguishable from that of the real-world hash function and its underlying permutation functions f_1 and f_2 . However, when the simulator responds to queries from the adversary $\mathcal{A}$, it may encounter “bad events”. The occurrence of a bad event signifies that the simulator cannot maintain this perfect mimicry, thereby enabling the adversary $\mathcal{A}$ to distinguish its environment. Our objective is to demonstrate that the total probability of all such bad events occurring is negligible.

The simulator $\mathcal{S}$ fundamentally operates by maintaining an internal directed graph $\mathcal{G}$, which records the entire history of queries made to the underlying permutation functions f_1 and f_2 . The nodes within this graph represent the internal states of the hash construction (e.g., the values of states A, B, and C generated during the message absorbing process), while the edges denote the input-output pairs of the permutation functions. Given the serial dependency in the construction from f_1 to f_2 , the structure of graph $\mathcal{G}$ becomes more intricate, necessitating that it simultaneously reflect queries to both f_1 and f_2 , as well as the transitions of states between them. In the following, we define bad events which ensure that the simulator does not run into these problems.

4.3.1 Structural Bad Events

These events primarily define undesirable topological anomalies within the graph structure $\mathcal{G}$. They ensure the unidirectional growth and uniqueness of the graph.

1. **BadNode:** This event is triggered when the simulator generates a new node N_{new} that is identical to an existing node in the graph across its full state (A, B, C) . This detection occurs before the simulator formally merges the new edge and node into the *Rooted* set. It is designed to prevent unintended cycles or state overlaps within the graph, ensuring that every path originating from the initial state remains unique.
2. **Double2Col:** This event describes the occurrence of two independent “fresh” collisions within a single query. It typically involves two or more message paths colliding on different state blocks, which probabilistically requires two independent random values to “hit” their targets. To simplify the proof, such complex structures requiring double collisions are directly labeled as bad events.
3. **3Col:** This event refers specifically to a sequential three-way collision on a single state block. It is triggered when the simulator finds that a new node, at a specific block $(A, B, \text{ or } C)$, collides with a set of nodes S_{coll} that are already involved in a two-way collision. By excluding such three-way overlaps, the simulator ensures that the size of the active set $\mathcal{C}_{active}$ is strictly bounded, thereby guaranteeing the uniqueness of $\mathcal{RO}$ constraints.

4.3.2 Logical Consistency Bad Events

These events describe the moments when the simulator is forced to violate specific consistency requirements.

1. **Inconsistency:** This occurs when a target value Y_{target} calculated by the simulator conflicts with an existing record Y_{stored} in the permutation table. This covers two critical scenarios:
 - (a) **“New Path vs. Old Table”:** The simulator attempts to assign a value to a new path, only to find the position was pre-occupied by the adversary via an inverse query, making it impossible to satisfy $\mathcal{RO}$ constraints.
 - (b) **“New Path vs. New Path”:** When a capacity collision causes two different message paths to point to the same input X . Since $\mathcal{RO}$ imposes independent output requirements for these two paths, the simulator faces irreconcilable mutual constraints.
2. **AUX^{bij}:** This event is triggered when an output value Y , forced by the simulator to maintain consistency, conflicts with other existing input/output pairs in the table, thereby violating the bijectivity of the permutation. This ensures that while the simulator satisfies the $\mathcal{RO}$ ledger, it does not produce self-contradictions at the underlying mapping level.

4.3.3 Complexity and Chain-Reaction Bad Events

These events serve to bound the simulator’s running time (to polynomial complexity) and the length of dependency chains, thereby preventing the probability analysis from becoming intractable.

1. **ThreeRounds:** This event is triggered when the recursive depth of `UpdateGraph` triggered by a single query reaches 3 or more ($depth \geq 3$). This ensures the simulator

terminates within polynomial time and limits the complexity of the dependency chains. Under favorable transcription conditions, the recursion depth naturally stays within 2.

2. **AUX^{fan}**: During recursive graph updates, if a new node unexpectedly activates multiple previously isolated “wild entries”, causing the graph structure to branch out in multiple directions, this event is triggered. It ensures that the state graph’s scale grows linearly relative to the number of queries and prevents the non-linear expansion of dependency chains.
3. **AUX^{inv}**: This event describes a scenario during an inverse query where the generated random preimage X_{rand} unexpectedly connects to a complex node already within the collision set S_{coll} . By excluding such inverse “hits” on complex structures, the simulator ensures that subsequent forced completions do not trigger uncontrolled chain reactions, thus maintaining a bounded recursion depth.

4.3.4 Interpretation

In the indistinguishability proof for the hash construction $\mathcal{H}^{f_1, f_2}$, we introduce a series of bad events to delineate the boundaries of perfect simulation. These events represent scenarios where the simulator $\mathcal{S}$ cannot simultaneously satisfy the consistency of the Random Oracle $\mathcal{RO}$, the bijectivity of the underlying permutations, and the uniqueness of the graph structure. The core of the proof lies in demonstrating that the total probability of all such events is negligible, thereby ensuring that the adversary $\mathcal{A}$ cannot distinguish between the simulated and real environments.

The simulator $\mathcal{S}$ maintains a directed graph $\mathcal{G}$ to track the query history of f_1 and f_2 . Bad events are categorized into three levels based on their impact on the simulation:

Structural Anomalies: BadNode acts as a safeguard against full-state collisions, preventing unintended cycles or path overlaps that would violate the tree-like growth of the rooted graph. **Double2Col** and **3Col** capture multi-path collisions on individual state blocks. By bounding these, we ensure that the size of the active candidate set $\mathcal{C}_{active}$ remains small, which is a prerequisite for managing $\mathcal{RO}$ constraints.

Logical Consistency: The Inconsistency event is the most critical addition, representing a logical deadlock where a calculated target value Y_{target} (required by $\mathcal{RO}$ consistency) conflicts with a pre-existing entry Y_{stored} in the permutation table. This covers both adversarial “pre-occupation” via inverse queries and internal collisions between different message paths. Similarly, **AUX^{bij}** ensures that these forced assignments do not violate the fundamental bijectivity of the permutation.

Complexity Control: The **AUX** family (**AUX^{inv}**, **AUX^{fan}**) and **ThreeRounds** serve to bound the simulator’s computational effort. **AUX^{inv}** and **AUX^{fan}** prevent the graph from expanding non-linearly or triggering uncontrolled chain reactions. Consequently, we can prove that on a “good” transcript, the recursion depth of **UpdateGraph** naturally satisfies $depth \leq 2$. Thus, the probability of **ThreeRounds** can be bounded by the union of other structural and auxiliary events.

In summary, the overall failure probability of the simulation is controlled via a union bound:

$$\text{BAD} := \text{BadNode} \vee \text{Double2Col} \vee \text{3Col} \vee \text{Inconsistency} \vee \text{AUX}^{inv} \vee \text{AUX}^{fan} \vee \text{AUX}^{bij}.$$

4.4 World Decomposition

In our indistinguishability analysis, we abstract the system into several worlds (see Fig. 2): the ideal world W_I , the real world W_R , and two intermediate worlds W_{IM1} and W_{IM2} , which serve to gradually bridge the ideal and real worlds.

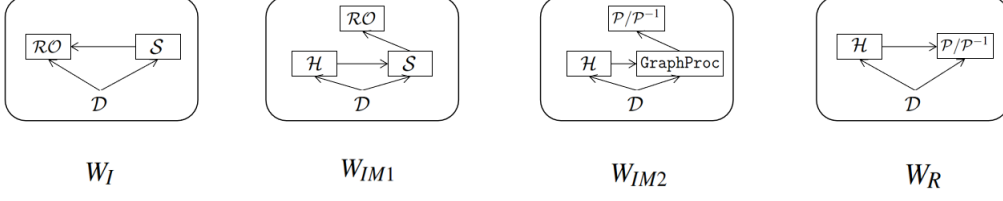


Figure 2: Definition of the different worlds. $\mathcal{P}$ returns responses with lazy sampling.

Ideal World W_I consists of a **random oracle** ($\mathcal{RO}$) and a **simulator** $\mathcal{S}$ (with forward and inverse interfaces). In this world, the **adversary** $\mathcal{D}$ has free access to both the $\mathcal{RO}$ to query hash values and the simulator $\mathcal{S}$ to query the underlying permutations. The core role of the simulator $\mathcal{S}$ is to mimic the behavior of the real-world underlying permutations. It achieves this by calling the $\mathcal{RO}$ to generate and return consistent responses that answer the adversary's queries on the underlying permutations. In this model, all operations are defined by random oracle, and there is no direct access to the real underlying permutations.

Real World W_R contains the hash function $\mathcal{H}$, constructed from the real permutations f_1, f_2 , as well as their inverses. All responses are provided via *lazy sampling* from the real permutations. The iterative mechanisms present in the simulator do not exist in W_R , making simulator-defined bad events (e.g., **ThreeRounds**) inapplicable.

To analyze the indistinguishability from W_I to W_R , we introduce two intermediate worlds. **Intermediate World 1** (W_{IM1}) allows access to the simulator's version of the hash function $\mathcal{H}^{\mathcal{S}}$ and the simulator interfaces, while the simulator still relies on the random oracle rather than the real permutations. In W_{IM1} , the random oracle $\mathcal{RO}$ is internal: only the simulator $\mathcal{S}$ may query it, the distinguisher has no direct access. By introducing W_{IM1} , we can separate the simulator's dependence on the random oracle (ensuring query consistency) from its internal table and node generation logic (the quality of internal randomness), thereby simplifying the indistinguishability analysis. **BAD** events remain meaningful in this world.

However, the difference between W_{IM1} and W_R is more challenging to handle because W_R relies entirely on lazy sampling from the real permutations and lacks the notion of iterations, preventing direct use of simulator-defined bad events. To address this, we define **Intermediate World 2** (W_{IM2}), which uses the real permutations to provide all answers while adding an auxiliary module **GraphProc**. **GraphProc** simulates the simulator's internal behavior during query processing, including node collection, path construction, and consistency enforcement. **GraphProc** does not modify the outputs of the real permutations; it merely reproduces the simulator's internal logic, including historical correlation and bad event detection. Consequently, W_{IM2} is output-wise identical to W_R but retains the simulator's internal structure, allowing us to compare the two worlds more effectively.

Let D^{W_x} symbolize the distinguisher's state in world W_x , with $x \in \{I, I_{M1}, I_{M2}, R\}$. Our aim is to quantify the indistinguishability advantage $\text{Adv}_{\mathcal{H}^{\mathcal{P}}, \mathcal{RO}}^{\text{indif}}(\mathcal{D})$. By applying the triangle inequality, this quantity can be broken down into:

$$\text{Adv}_{\mathcal{H}^{\mathcal{P}}, \mathcal{RO}}^{\text{indif}}(\mathcal{D}) = |\Pr[\mathcal{D}^{W_I} = 1] - \Pr[\mathcal{D}^{W_R} = 1]| \quad (1)$$

$$\leq |\Pr[\mathcal{D}^{W_I} = 1] - \Pr[\mathcal{D}^{W_{IM2}} = 1]| \quad (1a)$$

$$+ |\Pr[\mathcal{D}^{W_{IM2}} = 1] - \Pr[\mathcal{D}^{W_R} = 1]|. \quad (1b)$$

For the distance in (1a):

$$\begin{aligned}
(1a) &= |\Pr(D^{W_I} = 1 \wedge \text{BAD}) + \Pr(D^{W_I=1} \wedge \neg \text{BAD}) \\
&\quad - \Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD}) + \Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD}) \\
&\quad - \Pr(D^{W_{IM2}} = 1 \wedge \text{BAD}) - \Pr(D^{W_{IM2}} = 1 \wedge \neg \text{BAD})| \\
&\leq |\Pr(D^{W_I} = 1 \wedge \text{BAD}) - \Pr(D^{W_{IM2}} = 1 \wedge \text{BAD})| \\
&\quad + |\Pr(D^{W_I} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD})| \\
&\quad + |\Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM2}} = 1 \wedge \neg \text{BAD})| \\
&\leq |\Pr(D^{W_I} \text{ sets BAD})\Pr(D^{W_I} = 1|\text{BAD}) \\
&\quad - \Pr(D^{W_{IM2}} \text{ sets BAD})\Pr(D^{W_{IM2}} = 1|\text{BAD})| \tag{2a}
\end{aligned}$$

$$+ |\Pr(D^{W_I} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD})| \tag{2b}$$

$$+ |\Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM2}} = 1 \wedge \neg \text{BAD})|. \tag{2c}$$

The distance in (2a) can be bounded as follows:

$$(2a) \leq \max\{\Pr(D^{W_I} \text{ sets BAD}), \Pr(D^{W_{IM2}} \text{ sets BAD})\}. \tag{3}$$

In summary, we conclude that $\text{Adv}_{\mathcal{H}^P, \mathcal{RO}}^{\text{indif}}(\mathcal{D})$ (as defined in Equation (1) is ultimately bounded by the sum of the terms presented in (3), (2b), (2c), and (1b):

$$\text{Adv}_{\mathcal{H}^P, \mathcal{RO}}^{\text{indif}}(\mathcal{D}) \leq \max\{\Pr(D^{W_I} \text{ sets BAD}), \Pr(D^{W_{IM2}} \text{ sets BAD})\} \tag{4a}$$

$$+ |\Pr(D^{W_I} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD})| \tag{4b}$$

$$+ |\Pr(D^{W_{IM1}} = 1 \wedge \neg \text{BAD}) - \Pr(D^{W_{IM2}} = 1 \wedge \neg \text{BAD})| \tag{4c}$$

$$+ |\Pr[\mathcal{D}^{W_{IM2}} = 1] - \Pr[\mathcal{D}^{W_R} = 1]|. \tag{4d}$$

We will discuss these two probabilities and three distances below. In the following, we will use q to represent the total number of queries made by the distinguisher.

The remainder of this section will analyze the upper bound for each advantage term in Equations (4a)(4d) one by one. We will summarize the bound for each term and provide a brief justification, while deferring the detailed probability computations to Section 5.

- **Term (4a):** This is the total probability of the **BAD** events. Its upper bound will be computed in Section 5.1.
- **Term (4b):** This is the distance between W_I and W_{IM1} (which we can call the “interface hop”). We will argue that this term is 0.
- **Term (4c):** This is the distance between W_{IM1} and W_{IM2} (i.e., the “ $\mathcal{RO}$ vs. Real Primitive” hop). We will use the “bucket sampler” lemma from Section 5.3 to bound it.
- **Term (4d):** This is the distance between W_{IM2} and W_R . We will argue that this term is also 0.

Bad Events. As formalized in Lemma 3 (Section 5.1), the BAD events are proven to satisfy the upper bound

$$(4a) \leq \frac{216q^3}{(2^c - 4q)^2} + \frac{q^2 + q}{2^{2r+c+1}} + \frac{4q^2 - 4q}{2^{r+c}}.$$

Equivalence of Worlds W_I and W_{IM1} Conditioned on $\neg\text{BAD}$. In Section 5.2, by examining the internal structure of the simulator, we observe that as long as $\neg\text{BAD}$ holds, the simulator maintains internal consistency: each new query is correctly linked to prior queries, forming a valid hashing path and avoiding any conflicts or inconsistencies. Specifically, $\mathcal{RO}$ queries complete their hashing operations without conflicts, and underlying function queries are successfully associated with existing records via the **GraphProc** module.

Based on this internal consistency, it follows that under $\neg\text{BAD}$, the simulator's responses are indistinguishable from the real construction. Therefore, for any probabilistic polynomial-time (PPT) distinguisher, the ideal world W_I and the intermediate world W_{IM1} are indistinguishable.

Consequently,

$$(4b) = 0.$$

Equivalence of Worlds W_{IM1} and W_{IM2} Conditioned on $\neg\text{BAD}$. In Section 5.3, we analyze the two-stage primitives under the event $\neg\text{BAD}$. By reducing each stage to a *bucket sampler* and applying Lemma 5, we bound the distinguishing advantage between W_{IM1} and W_{IM2} . Each stage contributes at most $O\left(\frac{q^{3/2}}{2^c - O(q)}\right)$, and combining both stages gives a total bound:

$$(4c) \leq \frac{6q^{3/2}}{2^c - 4q}.$$

Equivalence of Worlds W_{IM2} and W_R . By construction, all queries to the real permutations f_1, f_2 in W_{IM2} are answered via lazy sampling. The **GraphProc** module updates internal tables, forms absorbing paths, and detects potential bad events, but it **does not modify the outputs** returned by the permutations. Therefore, from the perspective of any distinguisher $\mathcal{D}$, the queries and corresponding outputs in W_{IM2} are identical to those in W_R .

Combining this with the equivalence principle of lazy sampling (on-demand generation of outputs is equivalent to pre-drawing the entire permutation), we conclude that for any probabilistic polynomial-time (PPT) distinguisher, W_{IM2} and W_R are indistinguishable.

Consequently,

$$(4d) = 0.$$

Final Indifferentiability Bound. Combining the bounds derived for each term in (4a)–(4d), we obtain an explicit upper bound on the indifferentiability advantage:

$$\begin{aligned} \text{Adv}_{\mathcal{H}^P, \mathcal{S}}^{\text{indif}}(\mathcal{D}) &\leq (4a) + (4b) + (4c) + (4d) \\ &\leq \frac{216q^3}{(2^c - 4q)^2} + \frac{6q^{3/2}}{2^c - 4q} + \frac{q^2 + q}{2^{2r+c+1}} + \frac{4q^2 - 4q}{2^{r+c}}. \end{aligned}$$

This bound gives a concrete quantitative measure of the maximum distinguishing advantage any probabilistic polynomial-time distinguisher can achieve against the constructed hash function with the simulator.

5 Probability Computation

In this section, we establish upper bounds for the three terms from equations (4a), (4b), (4c) in Section 4.4, and (4d) already proved in Section 4.4.

5.1 Bad Events Probability Analysis

Our goal in this subsection is to prove the bound for term (4a), culminating in Lemma 3. The proof is structured as follows:

1. We first define the **Trimmed Graph** and prove two critical structural lemmas about it: Lemma 1 (bounding the completion depth) and Lemma 2 (bounding the tree size).
2. Using these lemmas as prerequisites, we then formally prove Lemma 3. This proof involves first bounding the per-query probability for each individual bad event (Section 5.1.1) and then summing these probabilities to find the total bound (Section 5.1.2).

Definition 4 (Trimmed Graph). The Trimmed Graph G_τ is a dynamically generated subgraph based on the simulator's query history. It consists of the set of **Rooted** nodes and their associated edges that have been successfully integrated without triggering any **BAD** events.

- **Initialization:** The **Trimmed** set is initialized to contain only the initial state (IV_A, IV_B, IV_C) .

$$\text{Trimmed} \leftarrow \{(IV_A, IV_B, IV_C)\}.$$

- **Update Rule:** For any query i , a new node (and its connecting edge) is added to G_τ if and only if:
 1. The node is reachable from the initial state IV (i.e., its parent node is already in G_τ).
 2. The addition of this node (and the edge leading to it) does not trigger any **BAD** event.

In essence, the Trimmed Graph represents the set of rooted nodes that have been successfully integrated without triggering any bad events, thus excluding all conflicting nodes and their subsequent derivations.

Lemma 1 (Recursion Depth Bound). *Conditioned on the event $\neg\text{BAD}$ (i.e., no bad events have occurred in the past or during the current query), the **UpdateGraph** procedure in the simulator executes at most 2 recursive calls ($\text{Depth} \leq 2$) for any single primitive query.*

Proof. We prove this by contradiction. Assume that the simulator enters a third recursive step (Depth 3). We trace the execution flow to show that this assumption implies the occurrence of a **BAD** event, contradicting the premise.

Step 1: The Initial Query (Depth 1). The process begins when the simulator receives a query (either forward f or inverse f^{-1}) and adds a corresponding new node N_1 to the graph. If N_1 does not interact with existing graph components, the process terminates.

Step 2: The First Constraint (Transition to Depth 2). A recursive call is triggered if and only if N_1 imposes a consistency constraint that must be resolved. Under the assumption of $\neg\text{BAD}$, this can occur in exactly two scenarios:

1. *Forward Collision:* The output capacity of N_1 collides with an existing node's capacity. Since **3Col** is excluded, this must be a valid 2-collision ($|C_{\text{active}}| = 2$).
2. *Inverse Hit:* An inverse query derives an intermediate state that hits a non-collision node in the graph (a hit on a collision node would trigger AUX^{inv} , which is excluded).

In either scenario, the simulator must perform a second step generating a subsequent node N_2 to satisfy the consistency requirements.

Step 3: The Contradiction (Transition to Depth 3). If the process were to require a third step, it would imply that the node N_2 , which was generated specifically to resolve the first constraint, *itself* creates a new conflict. This means the output of N_2 collides with yet another node in the graph.

However, this sequence of events forms a forbidden structure:

- If N_2 collides with a distinct capacity value, it forms a collision chain, constituting a **Double2Col** event.
- If N_2 collides with the same capacity value as N_1 , it forms a multi-collision, constituting a **3Col** event.

Since the lemma assumes $\neg \mathbf{BAD}$, neither of these events can occur. Therefore, the simulator is never forced to enter Depth 3.

Combining the above 3 steps yields

$$\text{depth} \leq 2.$$

□

Lemma 2 (Upper Bound on the Trimmed Graph Size). *For any number of queries $i \in \{0, 1, \dots, q\}$, assuming no **BAD** event has occurred, the number of rooted nodes in the graph, denoted as $|\text{Trimmed}_i|$, is upper bounded by:*

$$|\text{Trimmed}_i| \leq 4i + 1.$$

Proof. We prove this lemma by induction on the number of queries i .

Base Case ($i = 0$): Before any queries are made, the **Trimmed** set contains only the single root node representing the initial state (IV_A, IV_B, IV_C) . Thus, $|\text{Trimmed}_0| = 1$, which satisfies the bound $4 \times 0 + 1 = 1$.

Inductive Step: Assume the bound holds for $i - 1$ queries. We consider the i -th query and analyze the maximum number of nodes it can add to the graph, based on the recursion depth bound of 2 (Lemma 1).

- **Case 1: Forward Query.** A forward query typically extends an existing path by adding 1 new node (Depth 1). If this triggers a collision (Depth 2), the simulator adds at most 1 additional node to resolve consistency. Thus, a forward query adds at most **2** nodes.
- **Case 2: Inverse Query.** An inverse query is more complex because it may produce a state that is initially disconnected from the root. To maintain the property that all nodes in G_τ are rooted (reachable from IV):
 1. The simulator defines the target node itself (1 node).
 2. The simulator may need to lazily sample a **predecessor node** to connect this new target to the existing graph (1 companion node).
 3. If this process triggers a consistency check (Depth 2), up to 2 additional nodes may be required to resolve constraints.

Summing these potential contributions yields a conservative upper bound of **4** nodes for the worst-case inverse query.

Since $\neg\mathbf{BAD}$ excludes fan-out behaviors ($\mathbf{AUX}^{fan}$), the branching factor is limited, and this worst-case sum is sufficient. Therefore:

$$|\mathbf{Trimmed}_i| \leq |\mathbf{Trimmed}_{i-1}| + 4 \leq 4i + 1.$$

This completes the induction. $\square$

Lemma 3. *Let $\mathcal{D}$ be any distinguisher that issues a total of q primitive and construction queries. Assume that $2^c - 4q > 0$, and $x \in \{I, IM2\}$. Then the probability that $\mathcal{D}$ causes a $\mathbf{BAD}$ event satisfies*

$$\Pr[\mathcal{D}^{W_x} \text{ sets } \mathbf{BAD}] \leq \frac{224q^3}{(2^c - 4q)^2} + \frac{8(q^2 + q)}{2^{r+c} - q} + \frac{8(q^2 + q)}{2^{2r+c} - q}.$$

Proof. To compute the probability of a $\mathbf{BAD}$ event, we introduce some intermediate variables. We define the state collision set S_{coll} as:

$$S_{\text{coll}} := \{u \in \mathbf{Trimmed} \mid \exists v \in \mathbf{Trimmed} \setminus \{u\} \text{ s.t. } C_{\text{out}}(u) = C_{\text{out}}(v)\}.$$

Let $N_{\text{coll}} := |S_{\text{coll}}|$ denote the random variable representing the number of such nodes (i.e., the current size of the set of 2-collisions, noting that 3-collisions are excluded by $\neg\mathbf{BAD}$).

By the law of total probability, for any world $x \in \{W_I, W_{IM2}\}$, the total probability of a $\mathbf{BAD}$ event can be decomposed as:

$$\Pr_x(\mathbf{BAD}) \leq \sum_{m=0}^{4q+1} \sum_{i=1}^q \Pr_x(\mathbf{BAD}_i \mid N_{\text{coll}} = m \wedge \neg\mathbf{BAD}_{i-1}) \cdot \Pr_x(N_{\text{coll}} = m). \quad (5)$$

The conditional probability upper bound for the i -th query, $\Pr[\mathbf{BAD}_i \mid \dots]$, is composed of two components:

1. **Direct Terms** (ρ_{dir}): Probabilities of events that are independent of m (e.g., a fresh collision between two non-colliding nodes).
2. **Dependency Terms** (ρ_{dep}): Probabilities of events that are proportional to the current collision set size m (e.g., a new node colliding with an existing collision pair).

Thus, the conditional probability can be bounded by $\rho_{\text{dir}} + m \cdot \rho_{\text{dep}}$. Substituting this into Equation (5):

$$\begin{aligned} \Pr[\mathbf{BAD}] &\leq \sum_{i=1}^q \sum_{m=0}^{4q+1} (\rho_{\text{dir}} + m \cdot \rho_{\text{dep}}) \cdot \Pr[N_{\text{coll}} = m] \\ &= \sum_{i=1}^q \left[\underbrace{\rho_{\text{dir}} \sum_m \Pr[N_{\text{coll}} = m]}_{=1} + \underbrace{\rho_{\text{dep}} \sum_m m \cdot \Pr[N_{\text{coll}} = m]}_{=E[N_{\text{coll}}]} \right] \\ &= \sum_{i=1}^q \rho_{\text{dir}} + E[N_{\text{coll}}] \cdot \sum_{i=1}^q \rho_{\text{dep}}. \end{aligned} \quad (6)$$

Through this decomposition, we transform the total probability bound into the sum of two parts: one dependent only on the total queries q (Direct Terms), and the other proportional to the expected collision set size $E[N_{\text{coll}}]$ (Dependency Terms).

5.1.1 Event-wise Probability Estimates

In this section, we compute the conditional probability of each bad event occurring during the i -th query, denoted as $\Pr[\mathbf{BAD}_i \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}]$. According to Lemma 2, the total number of nodes in the graph is bounded by $4i$. Consequently, all collision probabilities involving the capacity part C are evaluated relative to the remaining available space $2^c - 4q$.

1. **BadNode.** This event implies that a newly generated full-state node (S_{up}, S_{mid}, C) completely coincides with one of the $4i$ existing nodes in the graph. As this represents a random sampling collision within the space $2^{2r+c} - q$, and considering that a single query generates at most four new nodes, the probability is upper bounded by:

$$\Pr(\mathbf{BadNode}_i \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{16i}{2^{2r+c} - q}.$$

2. **Double2Col.** This event corresponds to the occurrence of two independent weak collisions during the processing of a single query. Specifically, among the four new nodes generated by the simulator, at least two distinct nodes must independently collide with the existing nodes in the graph (bounded by $4i$) in terms of their capacity. The number of ways to choose two target nodes from the $4i$ existing ones is approximately $\binom{4i}{2} \approx 8i^2$. Since each collision must be satisfied independently within the available space of $2^c - 4q$, the probability of this event is upper bounded by approximately:

$$\Pr(\mathbf{Double2Col}_i \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{8i^2}{(2^c - 4q)^2}.$$

3. **3Col.** This event refers to the scenario where the capacity part C' of a newly generated node from a single query hits one of the m existing collision nodes in the set S_{coll} . Since C' is approximately uniformly distributed within the available space of $2^c - 4q$, the probability of this event is proportional to the size of the current collision set. Considering that a single query introduces at most one primary target node for such collision detection, the probability is upper bounded by:

$$\Pr(\mathbf{3Col}_i \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{m}{2^c - 4q}.$$

4. **Inconsistency.** This category encompasses two subtypes corresponding to different collision domains:

- **Type A (Table Value Conflict):** This event occurs within the **permutation space** ($r + c$ bits) and covers two scenarios:
 - (a) **Path Reconstruction Conflict:** When the simulator computes a specific target output Y_{target} to satisfy $\mathcal{RO}$ consistency (e.g., during the squeezing phase), this value accidentally coincides with one of the approximately $4(i - 1)$ existing entries in the query table.
 - (b) **Isolated Query Hit:** When the simulator generates a random response Y for an isolated query, this value accidentally coincides with the corresponding part of one of the approximately $4(i - 1)$ nodes in the graph.

Combining these two scenarios, the total number of potential conflict targets for a single query is approximately $8(i - 1)$. Consequently, we estimate the upper bound of the conflict count as $8i$. Since the sampling is performed within the

permutation space of size 2^{r+c} relative to the occupied entries, the probability is upper bounded by:

$$p_A = \frac{8i}{2^{r+c} - q}.$$

- **Type B (Active Set Conflict):** This event constitutes a dependency term occurring in the **capacity space** (c bits). It indicates that the current query triggers the convergence of multiple logical paths involving known collision nodes. This covers two scenarios: either one of the four newly generated nodes hits S_{coll} in terms of capacity, or the capacity part of the query input directly matches an output in S_{coll} . Summing these five potential trigger points relative to the available capacity space, the probability is upper bounded by:

$$p_B = \frac{5m}{2^c - 4q}.$$

Combining these two cases, the total probability for the Inconsistency event is upper bounded by:

$$\Pr(\text{Inconsistency}_i \mid N_{coll} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{8i}{2^{r+c} - q} + \frac{5m}{2^c - 4q}.$$

5. **ThreeRounds.** This event corresponds to the recursion depth reaching three levels, which would violate the structural assumptions of our simulator. This implies a chain reaction: first, one of the four new nodes generated during the i -th query accidentally hits one of the i isolated table entries, forcing the simulator into the second level of recursion (depth 2). Subsequently, a completion node generated to resolve this first hit must collide again with the remaining entries in the table. Given that each step requires an independent capacity collision within the available space of $2^c - 4q$, the joint probability for this sequence is upper bounded by:

$$\Pr(\text{ThreeRounds}_i \mid N_{coll} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{4i^2}{(2^c - 4q)^2}.$$

6. **AUX^{bij}.** This event implies that a new assignment to the permutation tables would violate bijectivity (i.e., attempting to map a new input to an existing output, or mapping an existing input to a new output). Under Lemma 1 and assuming $\neg \mathbf{BAD}_{i-1}$, each query triggers at most four permutation calls. For each call, the simulator must ensure that neither the input nor the output conflicts with the history. Consequently, the i -th query involves checks against at most $8(i-1)$ logical constraints derived from previous queries. This describes a collision within the 2^{r+c} -sized permutation space. The probability that the i -th query's assignment conflicts with the history is upper bounded by:

$$\Pr(\text{AUX}_i^{\text{bij}} \mid N_{coll} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{8i}{2^{r+c} - q}.$$

7. **AUX^{inv}.** This event implies that, during the processing of an *inverse* query, the capacity part of the lazily sampled internal state coincides with a node in the existing collision set S_{coll} .

Conditioned on $N_{coll} = m$ and $\neg \mathbf{BAD}_{i-1}$, considering that the capacity part is sampled uniformly from the available space, the probability is bounded by:

$$\Pr(\text{AUX}_i^{\text{inv}} \mid N_{coll} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{m}{2^c - 4q}.$$

8. **AUX**^{fan}. This event implies that, at the second level of recursion, a single newly generated trigger node simultaneously satisfies the connection conditions for two or more isolated entries in the query table, leading to an uncontrollable branching (fan-out > 1).

According to Lemma 1, each query introduces at most four new nodes into the graph, each acting as a potential trigger source. For each source, the event occurs if its capacity part simultaneously collides with a pair of candidate entries among the $\binom{i}{2} \approx i^2/2$ possible pairs in the table. As this requires two independent collisions against two independently distributed entries, the probability for the i -th query is upper bounded by:

$$\Pr(\mathbf{AUX}_i^{\text{fan}} \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{2i^2}{(2^c - 4q)^2}.$$

5.1.2 Summation and Final Bound

Based on the individual probability estimates, the total conditional probability of a **BAD** event during the i -th query is:

$$\Pr(\mathbf{BAD}_i \mid N_{\text{coll}} = m \wedge \neg \mathbf{BAD}_{i-1}) \leq \frac{14i^2}{(2^c - 4q)^2} + \frac{7m}{2^c - 4q} + \frac{16i}{2^{r+c} - q} + \frac{16i}{2^{2r+c} - q}$$

The next step is to use this expression to compute the total probability of a **BAD** event over the entire duration of the q queries. This will be achieved by summing over all possible values of i and m , thus yielding a final and comprehensive upper bound on the total probability of failure.

Thus, we have

$$\begin{aligned} \Pr_x(\mathbf{BAD} \mid N_{\text{coll}} = m) &\leq \sum_{i=1}^q \left(\frac{14i^2}{(2^c - 4q)^2} + \frac{7m}{2^c - 4q} + \frac{16i}{2^{r+c} - q} + \frac{16i}{2^{2r+c} - q} \right) \\ &\leq \frac{14}{(2^c - 4q)^2} \sum_{i=1}^q i^2 + \frac{7m}{2^c - 4q} \sum_{i=1}^q 1 + \frac{16}{2^{r+c} - q} \sum_{i=1}^q i + \frac{16}{2^{2r+c} - q} \sum_{i=1}^q i \\ &\leq \frac{7q(q+1)(2q+1)}{3(2^c - 4q)^2} + \frac{7mq}{2^c - 4q} + \frac{8q(q+1)}{2^{r+c} - q} + \frac{8q(q+1)}{2^{2r+c} - q} \end{aligned} \quad (8)$$

Plugging (8) into (6), we obtain

$$\begin{aligned} \Pr_x(\mathbf{BAD}) &\leq \sum_{m=0}^{4q+1} \Pr_x(N_{\text{coll}} = m) \cdot \left(\frac{7q(q+1)(2q+1)}{3(2^c - 4q)^2} + \frac{7mq}{2^c - 4q} + \frac{8q(q+1)}{2^{r+c} - q} + \frac{8q(q+1)}{2^{2r+c} - q} \right) \\ &= \left(\frac{7q(q+1)(2q+1)}{3(2^c - 4q)^2} + \frac{8q(q+1)}{2^{r+c} - q} + \frac{8q(q+1)}{2^{2r+c} - q} \right) \cdot \sum_{m=0}^{4q+1} \Pr_x(N_{\text{coll}} = m) \\ &\quad + \frac{7q}{2^c - 4q} \cdot \sum_{m=0}^{4q+1} m \cdot \Pr_x(N_{\text{coll}} = m) \\ &= \frac{14q^3 + 21q^2 + 7q}{3(2^c - 4q)^2} + \frac{8q^2 + 8q}{2^{r+c} - q} + \frac{8q^2 + 8q}{2^{2r+c} - q} + \frac{7q}{2^c - 4q} \cdot \mathbb{E}(N_{\text{coll}}). \end{aligned} \quad (9)$$

It now remains to compute the expected number of collision nodes, $\mathbb{E}(N_{\text{coll}})$. Based on our lemma that $|\text{Trimmed}| \leq 4q+1$, we define a Bernoulli variable V_i which is 1 if the node generated during the i -th query is a collision node, and 0 otherwise. By the linearity of

expectation, we have $E(N_{\text{coll}}) = \sum E(V_i)$. The probability that the i -th node is a collision node is bounded by the probability that its three parts collide with the parts of any of the $i - 1$ existing nodes.

$$\begin{aligned}
E(N_{\text{coll}}) &= \sum_{i=1}^{|\text{Trimmed}|} E(V_i) = \sum_{i=1}^{|\text{Trimmed}|} \Pr(V_i = 1) \\
&\leq \sum_{i=1}^{|\text{Trimmed}|} \frac{3(i-1)}{2^c - 4q} \\
&= \frac{3}{2^c - 4q} \sum_{i=1}^{|\text{Trimmed}|} (i-1) \\
&= \frac{3}{2^c - 4q} \cdot \frac{|\text{Trimmed}|(|\text{Trimmed}| - 1)}{2} \\
&\leq \frac{3}{2^c - 4q} \cdot \frac{(4q+1)(4q)}{2} \\
&= \frac{24q^2 + 6q}{2^c - 4q}. \tag{10}
\end{aligned}$$

Finally, plugging this upper bound for $E(N_{\text{coll}})$ from (10) into the total probability expression from (9), we obtain the final upper bound for the probability of a **BAD** event:

$$\begin{aligned}
\Pr_x(\text{BAD}) &\leq \frac{14q^3 + 21q^2 + 7q}{3(2^c - 4q)^2} + \frac{8q^2 + 8q}{2^{r+c} - q} + \frac{8q^2 + 8q}{2^{2r+c} - q} + \frac{7q}{2^c - 4q} \cdot \left(\frac{24q^2 + 6q}{2^c - 4q} \right) \\
&\leq \frac{518q^3 + 147q^2 + 7q}{3(2^c - 4q)^2} + \frac{8q^2 + 8q}{2^{r+c} - q} + \frac{8q^2 + 8q}{2^{2r+c} - q}.
\end{aligned}$$

In our proof, we assume that 2^c is much larger than $4q$. To obtain a clear and conservative upper bound that accounts for all lower-order terms in the numerator (observing that $\frac{518}{3}q^3 + 49q^2 + \frac{7}{3}q \leq 224q^3$ for any $q \geq 1$), we substitute the capacity-related terms with $224q^3$ and simplify the expression to:

$$\Pr_x(\text{BAD}) \leq \frac{224q^3}{(2^c - 4q)^2} + \frac{8(q^2 + q)}{2^{r+c} - q} + \frac{8(q^2 + q)}{2^{2r+c} - q}.$$

5.1.3 Security Discussion

Our overall bound consists of three main components: the capacity-related term $O(q^3/2^{2c})$, the primitive-related term $O(q^2/2^{r+c})$, and the full-state term $O(q^2/2^{2r+c})$. Since $2r+c > r+c$, the full-state term is strictly smaller and thus dominated by the primitive term. Therefore, the effective security of our construction is determined by the competition between the remaining two terms:

$$O(q^3/2^{2c}) \text{ and } O(q^2/2^{r+c}).$$

Their dominance is separated by a threshold. The security is bounded by $q \approx 2^{(r+c)/2}$ (from the primitive term) or $q \approx 2^{2c/3}$ (from the capacity term). The threshold occurs when these two query budgets are approximately equal:

- When $r \leq c/3$, the primitive term $O(q^2/2^{r+c})$ dominates the bound.

- When $r > c/3$, the capacity term $O(q^3/2^{2c})$ dominates, matching the double-sponge bound.

Therefore, in both scenarios, the security bound significantly exceeds the standard capacity-based birthday barrier of $O(q^2/2^c)$. By employing a serial construction with two underlying permutations of size 2^{r+c} , our scheme provides a robust security guarantee that effectively leverages the full state space. This result establishes a concrete security bound, demonstrating that the success probability of any generic attack is negligible up to the specified query limits. $\square$

5.2 Equivalence of Worlds W_I and W_{IM1} Conditioned on $\neg\text{BAD}$

This section is dedicated to proving that term (4b) from our world decomposition is equal to 0. We do this by proving the following lemma:

Lemma 4. *For any probabilistic polynomial-time (PPT) distinguisher $\mathcal{A}$,*

$$|\Pr[\mathcal{A}^{W_I} = 1 \wedge \neg\text{BAD}] - \Pr[\mathcal{A}^{W_{IM1}} = 1 \wedge \neg\text{BAD}]| = 0.$$

Proof. We first restate the expression from the lemma and apply the definition of conditional probability, $\Pr[A \wedge B] = \Pr[A \mid B] \cdot \Pr[B]$:

$$\begin{aligned} & |\Pr[\mathcal{A}^{W_I} = 1 \wedge \neg\text{BAD}] - \Pr[\mathcal{A}^{W_{IM1}} = 1 \wedge \neg\text{BAD}]| \\ &= |\Pr[\mathcal{A}^{W_I} = 1 \mid \neg\text{BAD}] \cdot \Pr[\neg\text{BAD}] - \Pr[\mathcal{A}^{W_{IM1}} = 1 \mid \neg\text{BAD}] \cdot \Pr[\neg\text{BAD}]| \\ &= \Pr[\neg\text{BAD}] \cdot (|\Pr[\mathcal{A}^{W_I} = 1 \mid \neg\text{BAD}] - \Pr[\mathcal{A}^{W_{IM1}} = 1 \mid \neg\text{BAD}]|). \end{aligned}$$

Next, we will show that the absolute difference term in the final expression (i.e., $|\Pr[\mathcal{A}^{W_I} = 1 \mid \neg\text{BAD}] - \Pr[\mathcal{A}^{W_{IM1}} = 1 \mid \neg\text{BAD}]|$) is equal to 0 by using a ‘‘Consistency Argument’’.

We argue that as long as no **BAD** event occurs, the simulator $\mathcal{S}$ will always maintain internal consistency. The simulator stores query history in two tables (`tablef1`, `tablef2`) and a state transition graph $\mathcal{G}$. The **GraphProc** module links new queries to existing records to form a valid hashing path. By definition, a **BAD** event occurs if and only if **GraphProc** either fails to establish a consistent link or encounters a conflict with existing records. Under $\neg\text{BAD}$, neither case arises, and every query has a unique consistent association with prior history.

- **RO queries:** The hashing process completes without conflict; all internal function calls are consistent with table entries.
- **Underlying function queries:** **GraphProc** successfully associates the query with existing records, yielding a complete hashing path.

Hence, under $\neg\text{BAD}$, all simulator responses are consistent with the accumulated query history, making its behavior indistinguishable from the real construction. Based on this consistency, the probability that the distinguisher outputs 1 in both worlds (conditioned on $\neg\text{BAD}$) is identical:

$$\Pr[\mathcal{A}^{W_I} = 1 \mid \neg\text{BAD}] = \Pr[\mathcal{A}^{W_{IM1}} = 1 \mid \neg\text{BAD}].$$

Substituting this result into our main expression, we get:

$$\Pr[\neg\text{BAD}] \cdot |0| = 0.$$

The lemma is proven. $\square$

5.3 Equivalence of Worlds W_{IM1} and W_{IM2} Conditioned on $\neg\mathbf{BAD}$

This section is dedicated to proving the bound for term (4c) from our world decomposition. We prove that, under the event $\neg\mathbf{BAD}$, the distinguishing advantage between games W_{IM1} and W_{IM2} can be upper bounded by a term of the form

$$O\left(\frac{q^{3/2}}{2^c - O(q)}\right).$$

Our proof strategy proceeds in two steps:

1. First, we analyze a *simplified model* a single primitive whose forward replies are generated by a “bucket sampler” and bound its distance from a true random permutation (Lemma 5).
2. Then, we use a sequence of game hops to reduce the comparison between W_{IM1} and W_{IM2} to two independent applications of Lemma 5. The resulting total bound is stated in Lemma 6.

For clarity, we model the total state as a triple $(A, B, C) \in \{0, 1\}^r \times \{0, 1\}^r \times \{0, 1\}^c$, where A, B are r -bit blocks and C is the c -bit *capacity part*. The two-stage primitives $f_1, f_2 : \{0, 1\}^{r+c} \rightarrow \{0, 1\}^{r+c}$ are independent random permutations, and the distinguisher may query them in both the forward and inverse directions. Let q be the total number of queries, and assume $q \leq 2^{c-2}$ so that the denominator in our bounds remains positive.

Lemma 5 (Chi-square bound for bucket sampler). *Let W_1 be the following sampling oracle: upon a forward query, it first picks a bucket index $k \in \{0, 1\}^r$ uniformly at random (the “outer r bits”), then samples uniformly without replacement from the 2^c elements inside bucket B_k (where B_k denotes the set of all $r + c$ bit values whose outer r bits equal k); inverse queries are answered by consistent lazy permutation sampling. Let W_2 be the uniform random permutation over $\{0, 1\}^{r+c}$ (both forward and inverse queries are sampled uniformly without replacement). If $q \leq 2^c$, then*

$$\text{Adv}_{\text{ind}}(W_1, W_2)(q) \leq \frac{1}{2} \cdot \frac{q^{3/2}}{2^c - 4q}.$$

For clarity, the notation $\text{Adv}_{\text{ind}}(W_1, W_2)(q)$ denotes the maximum indistinguishability advantage achieved by any PPT distinguisher making at most q total queries when attempting to differentiate between the system W_1 (the bucket sampler) and the system W_2 (the uniform ideal permutation). This is used as a shorthand for the supremum of the advantage over all possible distinguishers $\mathcal{D}$ making at most q queries.

Proof. Let $\Omega = \{0, 1\}^{r+c}$. For the i -th forward query, given history x^{i-1} , define

$$S(x) := \#\{\text{elements already drawn that share the same outer } r \text{ bits as } x\}.$$

Clearly $0 \leq S(x) \leq i - 1 \leq q$. The conditional probabilities in the two worlds are:

$$p_1(x) = \frac{1}{2^r} \cdot \frac{1}{2^c - S(x)}, \quad p_2(x) = \frac{1}{2^{r+c} - (i - 1)}.$$

The relative deviation is

$$\frac{p_1 - p_2}{p_2} = \frac{S(x) - \frac{i-1}{2^r}}{2^c - S(x)}.$$

Under $\neg\mathbf{BAD}$, we can conservatively bound $S(x) \leq 4q$ uniformly over all buckets, giving

$$\left| \frac{p_1 - p_2}{p_2} \right| \leq \frac{i - 1}{2^c - 4q}.$$

Substituting into the χ^2 definition, and noting that $\sum_x p_2(x) = 1$, we obtain

$$\chi^2(x^{i-1}) \leq \left(\frac{i-1}{2^c - 4q} \right)^2.$$

By the Dai–Hoang–Tessaro technique [DHT17],

$$\text{Adv}_{\text{ind}}(W_1, W_2)(q) \leq \left(\frac{1}{2} \sum_{i=1}^q \left(\frac{i-1}{2^c - 4q} \right)^2 \right)^{1/2}.$$

Since $\sum_{i=1}^q (i-1)^2 \leq q^3$, the bound simplifies to

$$\text{Adv}_{\text{ind}}(W_1, W_2)(q) \leq \frac{1}{2} \cdot \frac{q^{3/2}}{2^c - 4q}.$$

□

Lemma 5 bounds the advantage for the “single-stage bucket sampler vs. random permutation” setting. In our construction, the two stages f_1 and f_2 are independent under $\neg\mathbf{BAD}$ and both satisfy the premises of Lemma 5. Therefore, the distance between W_{IM1} and W_{IM2} can be decomposed into two terms one per stage each bounded via Lemma 5, and then combined.

Lemma 6. *Let $q := \sigma + q_{\mathcal{P}}$ be the total number of primitive queries after expanding all construction-level queries in W_{IM1} into calls to f_1, f_2 (including both forward and inverse queries). Assume $q \leq 2^{c-2}$. Then under $\neg\mathbf{BAD}$,*

$$|\Pr[D^{W_{\text{IM1}}} = 1 \mid \neg\mathbf{BAD}] - \Pr[D^{W_{\text{IM2}}} = 1 \mid \neg\mathbf{BAD}]| \leq \frac{6q^{3/2}}{2^c - 4q}.$$

Proof. Let W_a (resp. W_d) be the world obtained from W_{IM1} (resp. W_{IM2}) by removing the outer construction interface, leaving only the simulator plus the underlying primitives (resp. the random permutations). Let D_1 be the distinguisher obtained from D by expanding each construction-level query into the corresponding primitive queries to f_1 or f_2 . Let D_2 be the algorithm obtained from D_1 by performing locally the same “completion” steps (**GraphProc**) that the simulator performs internally, without altering any answers. Since the local **GraphProc** module in D_2 only reproduces the simulator’s internal logic without altering any answers, the distinguishing advantage of D_1 and D_2 is identical in any pair of worlds.

Since the construction layer is identical in both worlds,

$$\begin{aligned} & |\Pr[D^{W_{\text{IM1}}} = 1 \mid \neg\mathbf{BAD}] - \Pr[D^{W_{\text{IM2}}} = 1 \mid \neg\mathbf{BAD}]| \\ & \leq |\Pr[D_1^{W_a} = 1 \mid \neg\mathbf{BAD}] - \Pr[D_1^{W_d} = 1 \mid \neg\mathbf{BAD}]|. \end{aligned}$$

The sequence of intermediate worlds that appear in the proof of Lemma 6 is illustrated in Figure 3. Each transition replaces one component of the system while leaving the rest unchanged, thereby allowing us to bound the distinguishing advantage step by step.

In W_a , the simulator generates the outer r bits uniformly and freshly; under $\neg\mathbf{BAD}$ this can be replaced by an explicit bucket sampler G_s (one for each stage), which first selects a bucket index uniformly and then samples without replacement from the 2^c elements inside the bucket, with inverse queries answered consistently. Let W_b be the world obtained by making this replacement. The distributions of W_a and W_b are identical.

Now, we consider the distinguisher D_2 operating in these worlds. Under $\neg\mathbf{BAD}$, each initial primitive call in a stage can cause at most two further calls in the same stage, so each

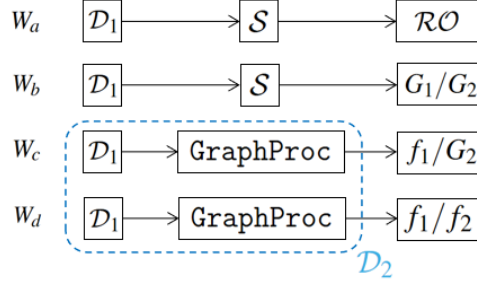


Figure 3: World decomposition: $W_a \rightarrow W_b \rightarrow W_c \rightarrow W_d$.

call is expanded by at most a factor of 3. Writing q_s for the number of stage- s primitive calls, we have $q_s \leq 3q$ for $s = 1, 2$. The index $s \in \{1, 2\}$ refers to the two sequential permutation calls in the absorbing step: $s = 1$ denotes all calls to f_1 , and $s = 2$ denotes all calls to f_2 . This separation allows us to independently bound the resource usage for each phase of the serial update. To uniformly cover possible overlaps of buckets between stages, reverse-direction occupancy, and the constant-factor expansion from GraphProc , we bound the remaining capacity in any bucket by $2^c - 4q$.

Define W_c to be the world obtained from W_b by replacing G_1 with a random permutation f_1 . Since the two stages are independent in W_b ,

$$\left| \Pr[D_2^{W_b} = 1 \mid \neg \mathbf{BAD}] - \Pr[D_2^{W_c} = 1 \mid \neg \mathbf{BAD}] \right| \leq \text{Adv}_{\text{ind}}(G_1, f_1)(q_1),$$

and similarly, replacing G_2 by f_2 yields a bound $\text{Adv}_{\text{ind}}(G_2, f_2)(q_2)$.

By Lemma 5, with the denominator replaced by the global bound $2^c - 4q$, we have

$$\text{Adv}_{\text{ind}}(G_s, f_s)(q_s) \leq \frac{1}{2} \cdot \frac{q_s^{3/2}}{2^c - 4q}.$$

Since $q_s \leq 3q$, it follows that $q_s^{3/2} \leq 3^{3/2} q^{3/2} \leq 6 q^{3/2}$, hence

$$\text{Adv}_{\text{ind}}(G_s, f_s)(q_s) \leq \frac{3 q^{3/2}}{2^c - 4q}.$$

Summing the contributions of the two stages and using the triangle inequality yields

$$\left| \Pr[D^{W_{\text{IM1}}} = 1 \mid \neg \mathbf{BAD}] - \Pr[D^{W_{\text{IM2}}} = 1 \mid \neg \mathbf{BAD}] \right| \leq \frac{6 q^{3/2}}{2^c - 4q}.$$

□

6 Tightness of the Bound

This section investigates the tightness of the security bound $\min\{2^{2c/3}, 2^{(r+c)/2}\}$ established in Theorem 1 for the Alternating Sponge. By presenting two distinguishing attacks, we show that the first term, $2^{2c/3}$, is tight. For the second term, however, we only obtain an attack with complexity $2^{(2c+r)/3}$, which does not fully match the bound $2^{(r+c)/2}$. These results show that the proven bound is tight in part, while also suggesting a remaining gap in the analysis of the second term.

6.1 Simple Birthday Attack

This attack targets the collision resistance of the Alternating Sponge itself, independent of any specific simulator design. The objective is to find a full-state collision: two distinct legitimate states that, upon absorbing specific message blocks, produce the exact same subsequent full state after a complete alternating permutation round (comprising f_1 and f_2).

Attack Steps and Constraint Analysis:

First, the attacker starts from the initialization vector IV and absorbs various random message prefixes to generate q distinct legitimate internal states (rooted nodes). Let any two such states be $N_1 = (S_{up,1}, S_{mid,1}, C_1)$ and $N_2 = (S_{up,2}, S_{mid,2}, C_2)$. The attacker then attempts to construct message blocks m_1 and m_2 for N_1 and N_2 , respectively, to force a collision.

Based on the serial absorption mechanism of the ASP construction, a full-state collision requires satisfying the input constraints of both permutations f_1 and f_2 sequentially:

Phase 1: Input Collision for f_1

To ensure identical outputs from f_1 , both the capacity and rate inputs must match:

$$C_1 = C_2 \quad (6.1)$$

$$S_{up,1} \oplus m_1 = S_{up,2} \oplus m_2 \implies m_2 = m_1 \oplus S_{up,1} \oplus S_{up,2} \quad (6.2)$$

Equation (6.1) imposes a c -bit constraint. However, because the attacker can freely determine m_2 using Equation (6.2) once m_1 is chosen, matching the rate part does not consume any collision probability. Consequently, the outputs of f_1 are guaranteed to be identical, i.e., $C'_1 = C'_2$ and $S'_{up,1} = S'_{up,2}$.

Phase 2: Input Collision for f_2

Since $C'_1 = C'_2$ is ensured by the previous phase, the capacity inputs to f_2 are naturally identical. The attacker only needs to match the rate inputs to f_2 :

$$S_{mid,1} \oplus \sigma(m_1) = S_{mid,2} \oplus \sigma(m_2) \quad (6.3)$$

Given that σ is a linear permutation function, Equation (6.3) can be rearranged and combined with the relation derived in Equation (6.2):

$$S_{mid,1} \oplus S_{mid,2} = \sigma(m_1 \oplus m_2) = \sigma(S_{up,1} \oplus S_{up,2}) \quad (6.4)$$

Equation (6.4) imposes an r -bit constraint.

Attack Complexity and Tightness:

To find a full-state collision, the attacker needs to find a pair of nodes among the q rooted nodes that satisfies the conditions in Equation (6.1) and Equation (6.4), which constitute a total of $c + r$ bits of constraints. The probability of finding such a pair is approximately $\mathcal{O}(q^2/2^{r+c})$.

When the number of queries reaches $q \approx 2^{(r+c)/2}$, the attacker can find a full-state collision with high probability, thereby breaking the hash construction. This demonstrates that the term $\frac{r+c}{2}$ in the theorem represents the structural upper bound of the ASP structure, meaning this aspect of the bound is tight with respect to the structural limit.

6.2 Simulator-Specific Attack

When $r > c/3$, the security bound established in Theorem 1 is dominated by the $\frac{2c}{3}$ term. To elucidate the origin of this theoretical bottleneck, we construct a distinguishing attack specifically targeting the logic of our simulator. This attack exploits the serial dependency of the ASP construction to forge a “chain hit” query path, forcing the simulator into a deadlock (i.e., triggering the **Inconsistency** or **BadNode** events) while attempting to maintain consistency with the $\mathcal{RO}$.

Attack Steps and Constraint Analysis:

The distinguisher $\mathcal{D}$ executes the following sequence of queries to induce an irreconcilable contradiction within the simulator:

First, the attacker performs blind primitive queries to pre-determine specific input-output pairs. The attacker makes q random forward queries to f_1 , obtaining the record set $\{X_{1,j} \rightarrow Y_{1,j}\}_{j=1}^q$, and q random forward queries to f_2 , obtaining the record set $\{X_{2,k} \rightarrow Y_{2,k}\}_{k=1}^q$.

Subsequently, starting from the initialization vector IV , the attacker normally absorbs random messages to expand the tree, generating q legitimate internal states (rooted nodes), denoted as $N_i = (S_{up,i}, S_{mid,i}, C_i)$ for $i \in \{1, \dots, q\}$.

Finally, the attacker attempts to find a perfect matching tuple (i, j, k) and a specific message block m among these three large datasets. The goal is to ensure that the serial processing of node N_i upon absorbing message m exactly hits the pre-queried records $X_{1,j}$ and $X_{2,k}$ sequentially.

This “chain hit” requires satisfying exceedingly strict physical constraints simultaneously:

Constraint 1: Hitting the First-Phase Permutation f_1

When node $N_i = (S_{up,i}, S_{mid,i}, C_i)$ absorbs the message block m , the full input fed into f_1 is $(S_{up,i} \oplus m) \parallel C_i$. For this processing step to strictly hit the pre-queried record $X_{1,j}$, the entire input must match exactly:

$$(S_{up,i} \oplus m) \parallel C_i = X_{1,j} \quad (6.5)$$

This macroscopic equality directly decomposes into two sub-constraints. First, it requires matching the capacity part:

$$C_i = \text{Cap}(X_{1,j}) \quad (6.6)$$

And matching the rate part:

$$S_{up,i} \oplus m = \text{Rate}(X_{1,j}) \implies m = S_{up,i} \oplus \text{Rate}(X_{1,j}) \quad (6.7)$$

Equation (6.6) imposes a c -bit constraint. Since the message m is uniquely determined by Equation (6.7) and can be freely chosen by the attacker, this step does not consume any additional collision probability.

Constraint 2: Serial Transfer and Hitting the Second-Phase Permutation f_2

Upon hitting f_1 , the capacity part of its output $Y_{1,j}$ will be directly fed as the input to f_2 . To subsequently hit the record $X_{2,k}$, the capacities must seamlessly connect:

$$\text{Cap}(Y_{1,j}) = \text{Cap}(X_{2,k}) \quad (6.8)$$

Equation (6.8) constitutes the second c -bit constraint.

Simultaneously, the rate input to f_2 must also coincide with the record $X_{2,k}$:

$$S_{mid,i} \oplus \sigma(m) = \text{Rate}(X_{2,k}) \quad (6.9)$$

By substituting the uniquely determined m from Equation (6.7) into Equation (6.9), we derive the ultimate rate constraint:

$$S_{mid,i} \oplus \sigma(S_{up,i} \oplus \text{Rate}(X_{1,j})) = \text{Rate}(X_{2,k}) \quad (6.10)$$

Equation (6.10) imposes an r -bit constraint.

Attack Complexity and Simulator Limit:

To successfully execute the aforementioned chain attack, the attacker must satisfy the conditions in Equation (6.6), (6.8), and (6.10), which amount to a rigorous constraint of

$c + c + r = 2c + r$ bits in total. Since the attacker possesses q valid nodes, q records for f_1 , and q records for f_2 , the combination space for the tuple (i, j, k) is q^3 .

Therefore, the probability of such a “chain hit” occurring is approximately $\mathcal{O}(q^3/2^{2c+r})$. Once this hit occurs, because all inputs and outputs along the path have been rigidly fixed by the attacker’s pre-computation queries, the simulator loses all degrees of freedom to adjust internal values to maintain $\mathcal{RO}$ consistency for the overall hash output. At this point, the simulator inevitably triggers a deadlock or aborts.

When the number of queries reaches $q \approx 2^{\frac{2c+r}{3}}$, the described attack breaks the simulator with high probability. Hence, $\frac{2c+r}{3}$ should be understood as an upper limit of the current simulator-based proof approach. By comparison, our theorem only proves security up to $\frac{2c}{3}$. Therefore, the proven bound is not tight with respect to this simulator, although the gap is only $\frac{r}{3}$ in the exponent. In this sense, the analysis is close to tight for the present proof framework, but not fully tight.

6.3 Discussion on the Threshold $r \approx \frac{c}{3}$

The two attacks presented above reveal the security bottlenecks of the ASP construction under different parameter configurations. These can be analyzed by examining the natural threshold $r \approx \frac{c}{3}$:

- **Case 1: $r \leq \frac{c}{3}$ (Constrained by Physical Limits).** In this parameter regime, $\frac{r+c}{2} \leq \frac{2c}{3}$, and the overall security bound is dominated by the $\mathcal{O}(q^2/2^{r+c})$ term. This perfectly aligns with the Simple Birthday Attack described in Section 6.1. It implies that the lower bound derived from our mathematical proof completely coincides with the theoretical physical upper limit of the structure itself. In this scenario, the proof is **tight**.
- **Case 2: $r > \frac{c}{3}$ (Constrained by Simulator Complexity).** For typical lightweight parameters, the bound is dominated by the capacity term $\mathcal{O}(q^3/2^{2c})$. Section 6.2 shows that the current simulator-based proof approach cannot go beyond $\frac{2c+r}{3}$, whereas our proof only establishes security up to $\frac{2c}{3}$. Therefore, the bound is **not fully tight** in this regime, and the remaining gap is $\frac{r}{3}$ in the exponent. Closing this gap would require tracking higher-degree inner collisions and partial extensions, which appears analytically intractable under current indistinguishability techniques.

7 Conclusion

This paper addresses the high memory cost of the double sponge construction in resource-constrained environments by proposing a new low-memory hashing mode, named **Alternating Sponge**. Our scheme employs a dual-permutation, dual-path design that re-organizes the state and alternates permutation calls to improve capacity utilization. This approach reduces the state size from $2r + 2c$ bits to just $2r + c$ bits, thereby lowering memory usage while maintaining strong diffusion.

For security, we provide a formal proof in the indistinguishability framework. Our analysis uses a tailored simulator and a series of game hops to bound the adversary’s advantage. We show that our construction offers security comparable to the original double sponge construction. Specifically, our security is bounded by both the capacity term ($\mathcal{O}(q^3/2^{2c})$) and the underlying primitive’s term ($\mathcal{O}(q^2/2^{r+c})$). This result exceeds the capacity birthday bound ($\mathcal{O}(q^2/2^c)$) found in traditional sponge functions.

This work offers a new paradigm for deploying high-security hash functions in resource-constrained environments. Future research directions include refining state update strategies, extending the construction to more versatile cryptographic primitives, and exploring compatibility with different underlying permutations.

The proposed ASP construction also exhibits potential for efficiency optimization in traditional instantiations. Inspired by designs that leverage the internal states of block ciphers to construct PRFs (e.g., the AES-based PRF design [MN17]), one possible direction is a “round-halving” instantiation strategy. Specifically, for a primitive like Keccak, which employs a 24-round underlying permutation $\mathcal{F}$, one may use its first 12 rounds to instantiate f_1 of the ASP, and its subsequent 12 rounds to instantiate f_2 . In this way, while processing a single message block m_i originally required invoking two full permutations (totaling 48 rounds of computation), it may instead require only a single full 24-round permutation to complete the alternating absorption of both m_i and $\sigma(m_i)$. If such an instantiation can be justified securely, it could significantly improve the throughput of the overall hash function.

In addition, unlike structures such as the AES-PRF where the full state is continuously transmitted, the alternating absorption mechanism of ASP provides a form of structural separation. At the junction between f_1 and f_2 , only the c -bit capacity remains continuous, whereas the r -bit rate part is interrupted and modified by the injection of the intermediate message $\sigma(m_i)$. This discontinuity in the internal state may hinder the propagation of differential or linear cryptanalysis characteristics across the full 24 rounds. Therefore, exploring such ASP instantiation schemes based on reduced-round permutations, and understanding the tradeoff between practical throughput and cryptanalytic resistance is an interesting direction for future research.

References

- [BDPA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche, editors, *ECRYPT Hash Workshop 2007*, pages 1–20, 2007.
- [BDPA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indistinguishability of the sponge construction. In Nigel P. Smart, editor, *Advances in Cryptology EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, Berlin, Heidelberg, 2008.
- [CDMP05] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-damgård revisited: How to construct a hash function. In Victor Shoup, editor, *Advances in Cryptology CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 430–448. Springer, Berlin, Heidelberg, 2005.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Submission to nist. NIST Lightweight Cryptography Selection, 2021. Official design document for the NIST LWC competition.
- [DHT17] Wei Dai, Viet Tung Hoang, and Stefano Tessaro. Information-theoretic indistinguishability via the chi-squared method. In *Advances in Cryptology — CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 497–523. Springer, 2017.
- [Dwo15] Morris J. Dworkin. Sha-3 standard: Permutation-based hash and extendable-output functions. Technical Report 202, Federal Information Processing Standards (NIST FIPS), 2015.
- [GKL⁺11] Praveen Gauravaram, Lars R. Knudsen, Gregor Leander, Florian Mendel, Christian Rechberger, Martin Schläffer, and Søren S. Thomsen. Grøstl – a SHA-3 candidate. Submission to NIST (Round 3), 2011. Official design document.

- [Gol01] Oded Goldreich. *Foundations of Cryptography, Volume 1: Basic Tools*. Cambridge University Press, 2001.
- [Hir06] Shoichi Hirose. Some plausible constructions of double-block-length hash functions. In Matthew Robshaw, editor, *Fast Software Encryption*, pages 210–225, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [LLHL26] Hongli Li, Changlun Li, Honggang Hu, and Fengmei Liu. Indifferentiability of the sponge hash family: New attacks and the optimal construction. *IACR Transactions on Symmetric Cryptology*, 2026(1):31–75, 2026.
- [LM24] Charlotte Lefevre and Bart Mennink. Permutation-based hashing beyond the birthday bound. *IACR Transactions on Symmetric Cryptology*, 2024 No. 1:71–113, 2024.
- [Mer89] Ralph C. Merkle. One way hash functions and des. In *Advances in Cryptology CRYPTO 89*, pages 428–446. Springer, 1989.
- [MN17] Bart Mennink and Samuel Neves. Optimal PRFs from blockcipher designs. *IACR Transactions on Symmetric Cryptology*, 2017(3):228–252, 2017.
- [MRH04] Ueli M. Maurer, Renato Renner, and Clemens Holenstein. Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In Moni Naor, editor, *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004*, volume 2951 of *Lecture Notes in Computer Science*, pages 21–39. Springer, 2004.
- [NSS21] Yusuke Naito, Yu Sasaki, and Takeshi Sugawara. Double-block-length hash function for minimum memory size. In *International Conference on Selected Areas in Cryptography*, pages 308–329. Springer, 2021.
- [Wu11] Hongjun Wu. The hash function JH. Submission to NIST SHA-3 competition, 2011. Round 3 submission.